CM3070 Computer Science Final Year Project

Machine Learning and Neural Networks - Deep Learning Breast Cancer Detection

Done by: Aesterin Huang Qiaorui

Date of submission: 28 September 2026

GitHub repository: https://github.com/hngnyng/UOL-FYP-2026

Abstract

This report is a continuation from the interim report, that depicts a detailed academic documentation of the Machine Learning Deep Learning Breast Cancer Detection project, as part of the Computer Science Final Project submission. The project's expectation is to revise the problem statement, design and develop a solution and evaluate the effectiveness of the implemented solution or technology in addressing the problem.

Abstract 2
Introduction (626 words) 6
Project Template 6
Project Background & Context 6
Project Motivation 6
Project Aims & Objectives 7
Project Scope 7
Project Requirements 7
Project Approach 7
2 Literature Review (2462 words) 10
History of breast cancer technology 10
Breast cancer screening process with mammography 13
Breast cancer screening technique 14
Deep Learning integration in mammography 16
Challenges and limitations of CNN 17
Challenges and limitations of Machine Learning/Deep Learning in breast cancer detection 17
Past projects on breast cancer detection using Machine Learning/Deep Learning 18
3 Design (1563 words) 21
Project Timeline 22
Code Pipeline 23
Dataset Description 24
Measure Of Success 27
CNN Model Architecture 28
Evaluation Protocol 32
4 Implementation (1126 words) 34
Importing and loading datasets 34
Explorative Data Analysis (EDA) 35
Resolving issues with mass/calcification train datasets 35

Binary label encoding 37
One-hot encoding 38
Split train, validation, test data 39
Pixel normalisation 41
Establish Baseline's accuracy 42
Converting input into tensors 42
Building the custom models 43
1. Defining the convnet architecture 43
2. Compile the model by selecting its loss function, optimizer, and evaluation metrics 45
3. Create callbacks and class weights 45
4. Fit the model using the training data and provide the validation data for performance monitoring 46
5. After training, plot the accuracy and loss graph, print the results of the performance metrics, and evaluate the final model once on the unseen test dataset using the confusion matrix 46
Building the ResNet50 & VGG16 model 50
1. Defining the convnet architecture 50
2. Compile the model by selecting its loss function, optimizer, and evaluation metrics 51
3. Create callbacks and class weights 52
4. Fit the model using the training data and provide the validation data for performance monitoring 52
5. After training, plot the accuracy and loss graph, print the results of the performance metrics, and evaluate the final model once on the unseen test dataset using the confusion matrix 53
5 Evaluation (1071 words) 56
Results 56
Custom CNN models 56
Pre-trained models 57
Current Limitation 59

6 Conclusion (351 words) 60
Reference 62
Appendices 68
Appendix A – Gantt chart 68
Appendix B – Project pipeline flowchart 68
Appendix C – FYP diagrams/recording slide 69
Appendix D – Measure of Success metrics description 69
Appendix E – CNN models design process 71
Appendix F.1 – Base CNN Model results 75
Appendix F.2 – Base + Data Augmentation Model results 76
Appendix F.3 – Base + Data Augmentation + L2 Regularisation Model results 77
Appendix F.4 – Base + Data Augmentation + Hyperparameter Tuning Model results 78
Appendix F.5 – Base + Data Augmentation + Hyperparameter Tuning Model results – on X_test 78
Appendix F.6 – Base + Data Augmentation + Hyperparameter Tuning Model results – on df_test 79
Appendix G.1 – ResNet50 base 80
Appendix G.2 – ResNet50 with best hyperparameters 81
Appendix G.3 – VGG16 83
Appendix G.4 – VGG16 with best hyperparameters 85

Introduction (626 words)

This section introduces the project to provide contextual knowledge that justifies the motivation to address the problem statement, outlines the plans, expectations, and strategy for executing the project.

Project Template

The project template chosen is **Machine Learning and Neural Networks - Deep Learning Breast Cancer Detection.**

Project Background & Context

In early 2025, the UK National Health Service (NHS) launched a major project that addresses the question on whether Deep Learning (DL) assisted X-ray mammography is reliable enough to achieve accurate results or improve the accuracy of breast cancer screening.

The research foresees that a successful outcome of this project can potentially have a DL system to replace at least two radiologists for faster diagnostic turn-around and the liberation of specialists for other tasks.

Project Motivation

The motivation of attempting this project comes from the fact that breast cancer is the most common disease affecting mainly women. Research has shown that breast cancer generally has a lower mortality rate due to early awareness and detection. This highlights the critical role of investment in advanced mammography technology for improving screening accuracy and early diagnosis. Through this project, not only can we discover a potential possibility to change the trajectory of advanced cancer screening, but this research also has the potential to optimise healthcare and staff resource, alleviate the healthcare system's burnout, allow specialist to redirected towards higher priority clinical tasks and subsequently improve the overall efficiency of the healthcare system.

Project Aims & Objectives

The aim of the project is to determine if DL assisted X-ray mammography can improve the accuracy of breast cancer screening, on predicting whether the breast tumours in question are benign or malignant.

The objective of the project is to create a Convolutional Neural Network (CNN) model that has statistical power and the capability to train the Digital Database for Screening Mammography (DDSM) dataset to perform predictions and evaluate the model's predictive performance using appropriate classification metrics.

Project Scope

The project scope targets radiologists and medical professionals responsible for handling and interpreting mammograms. The project is limited to binary classification of mammogram images as benign or malignant using the DDSM dataset. The scope does not extend to actual clinical deployment or any integration with existing hospital mammography practices or systems. The model will be developed and evaluated in a local environment without access to dedicated GPUs which may impose constrains on the model complexity.

Project Requirements

To achieve the aims and objectives, the project is required to fulfil the following:

- Working CNN with minimum statistical power to train the DDSM dataset
- Regularised CNN networks that adhere strictly to the machine learning workflow
- Develop a competitive model with DL models referenced in the *Deep Learning with Python* textbook by Chollet

Project Approach

The approach taken for the project follows the universal machine learning workflow, based on the textbook *Deep Learning with Python*.

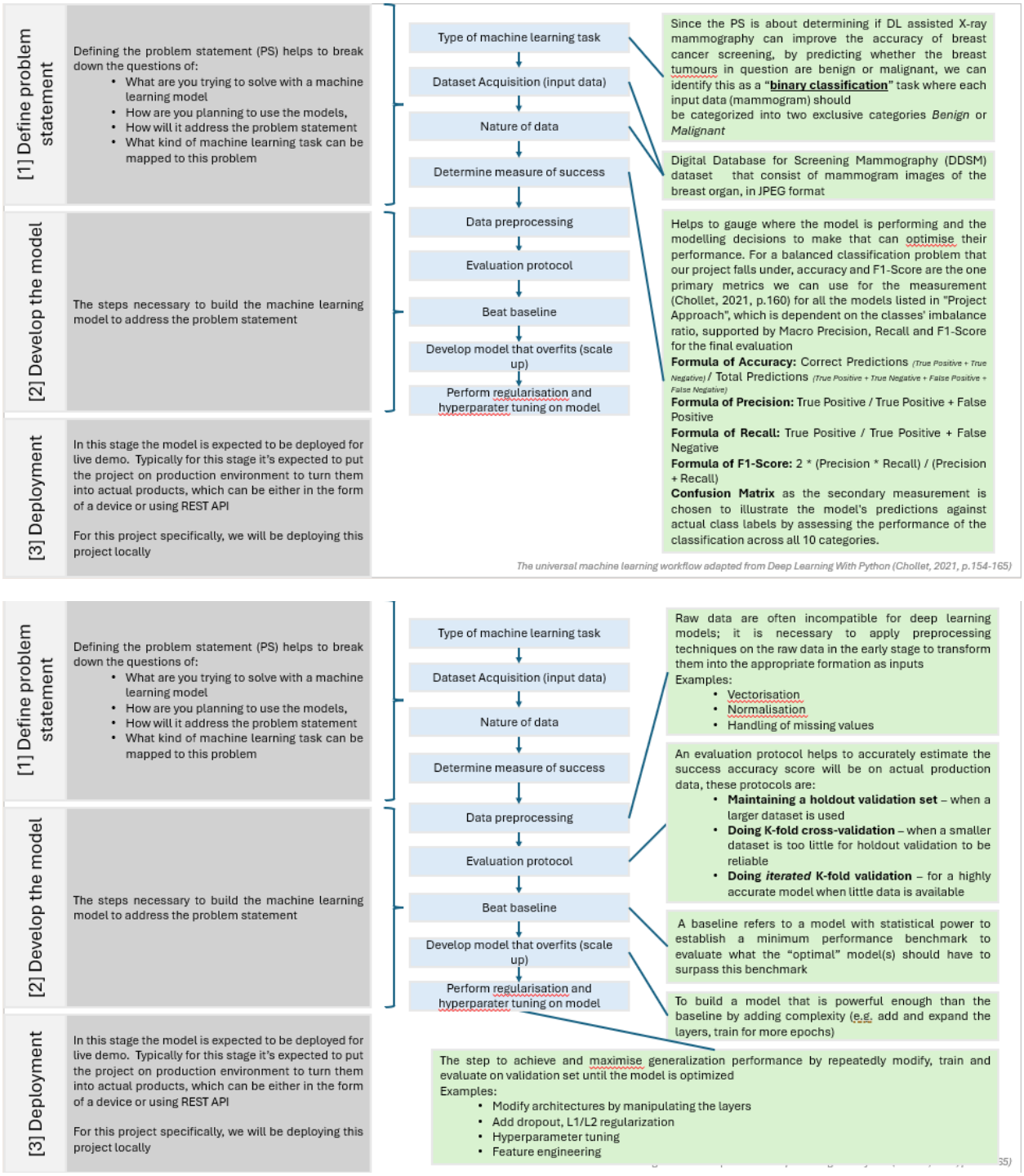


Figure 1 - The universal machine learning workflow adapted from Deep Learning with Python (Chollet, 2021, p.154-165) (Appendix C)

The workflow involves the usage of Computer science techniques and processes that include the fundamentals of artificial neural networks, dataset splitting, model building with TensorFlow and training and testing of the model.

We utilise a series of CNN models that are custom built and pre-trained to evaluate the effectiveness of addressing the problem statement. For the custom base CNN model with minimum statistical power, the following methods will be applied to improve its generalisation capability:

- **Data strategy (data augmentation)** – by increasing variety of the training dataset and improve the model's ability to generalise unseen mammogram data
- **Regularisation** – for reducing overfitting using techniques such as dropout and weight regularisation
- **Hyperparameter** – by tuning the hyperparameter to find the best parameter combination that can optimise the model's performance (e.g. batch size, learning rate, number of filters, etc)
- **Utilise transfer learning** techniques using pre-trained CNN models to counter the limitations of the custom CNN models

2 Literature Review (2462 words)

History of breast cancer technology

In the early days before mammography became the blueprint for breast cancer detection, there was no specific technology that was cultivated to perform this procedure. Breast cancer detection started in the early 1900s, when X-ray imaging was discovered as a form of radiation therapy to treat cancer (A Brief History of Cancer, n.d.). Subsequently, in the 1960s, through extensive research and fundings, mammography was discovered to be an appropriate screening tool for breast cancer detection, and eventually widely accepted method among the medicinal field by the federal agent the Food and Drug Administration (FDA) (Nass et al., 2001).

Despite the widespread adoption of mammography during this era, the early systems remained unreliable by modern standards. This was due to primitive imaging hardware and lack of standardised processing protocols, including the lack of older female demographic data, that resulted in high false-positive rates and low specificity. Furthermore, the technology was limited to consistently produce high-quality mammograms and it lack the ability to smartly detect and identify all forms of breast cancers, in all kinds of breast density, a specialised doctor or radiologists are critically essential to make the diagnosis (Nass et al., 2001).

The mammography technology and technique continue to make incremental improvements over the years to overcome the limitations that legacy technology could not handle. Early mammograms rely on 2-dimensional X-ray imaging, which can be described as outdated since the image quality was deficient and unreliable for identifying signs or symptoms of cancer. The current standard for imaging quality uses the Digital Breast Tomosynthesis (DBT) technology to overcome the limitations of 2D X-ray mammograms, improving the accuracy for breast cancer detection. DBT targets multiple angles of the breast to generate 3-dimensional X-ray images, effective for screening breasts with dense or overlapping tissues that may be harder to detect with older technologies (Wang L, 2024).

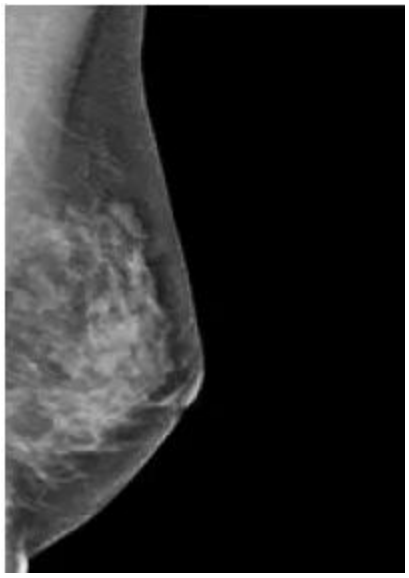


Figure 2 - Mammogram in the 1960s – (Joe, n.d.)

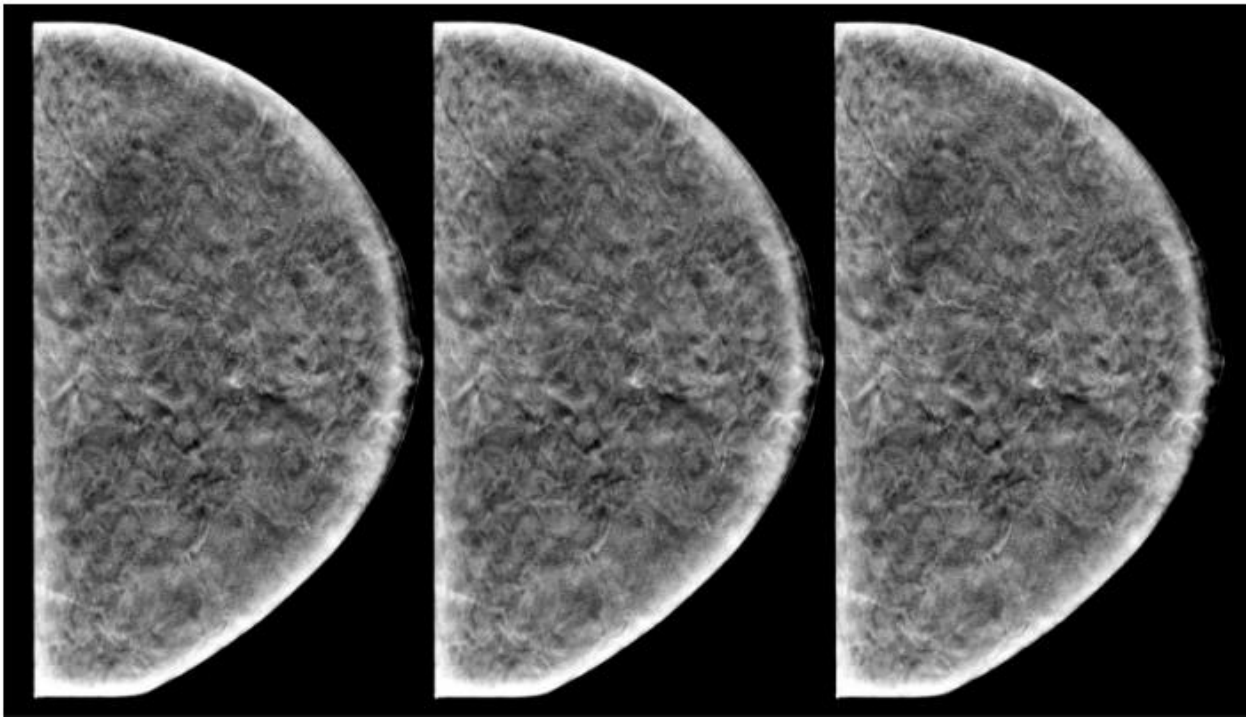


Figure 3 - DBT mammogram with simulated grouped microcalcifications (n.d.)

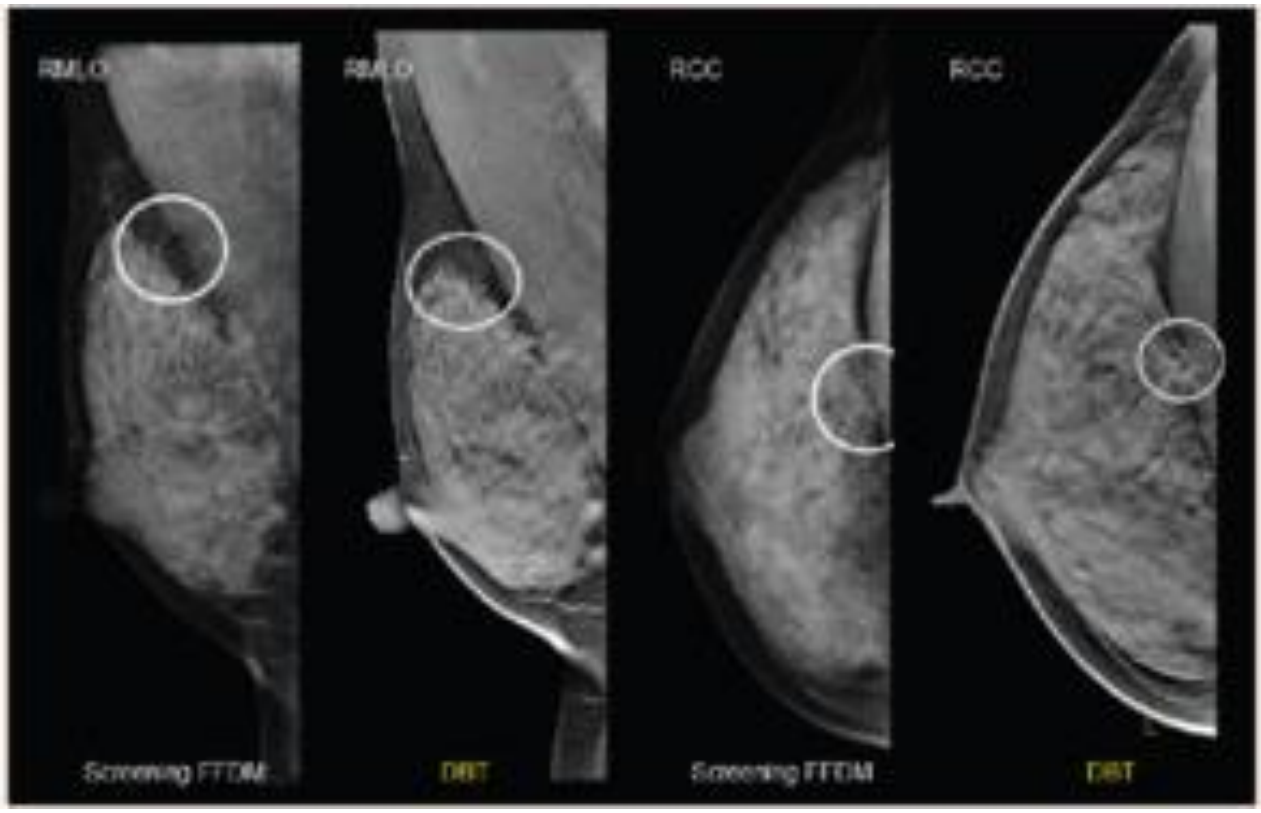


Figure 4 - DBT mammogram (Liew, n.d.)

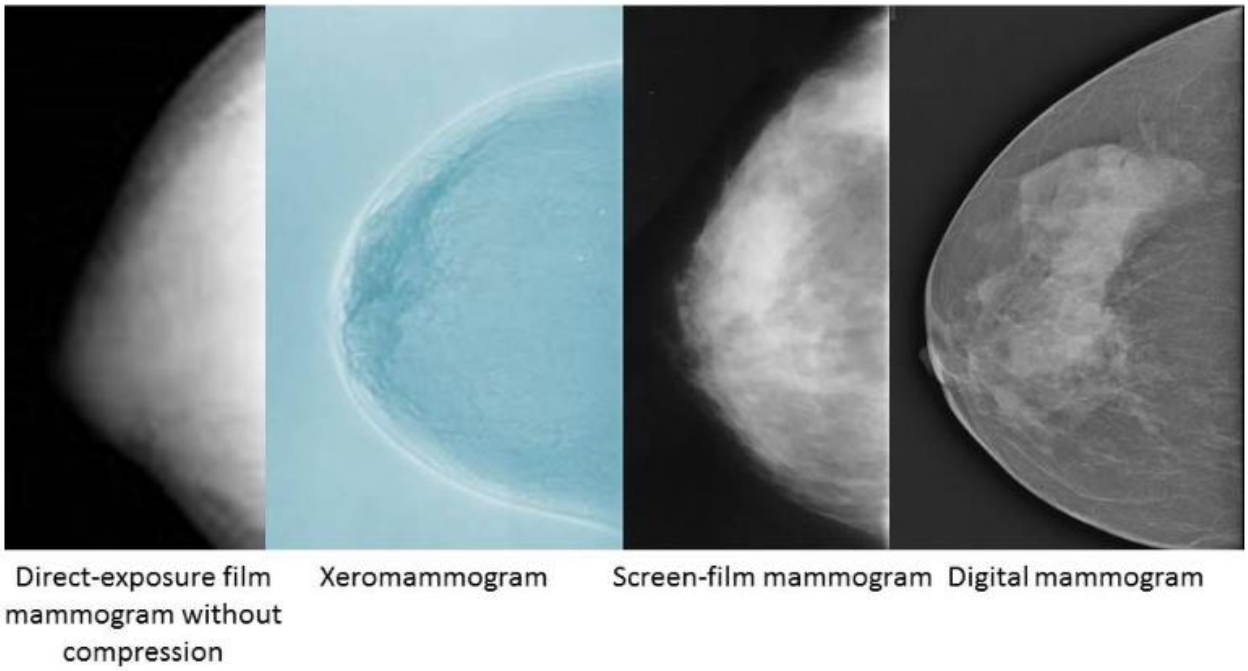


Figure 5 - The evolution of breast imaging techniques from 1965 to the present (n.d.)

The continuous revolution of mammography serves as a foundational movement to substantial improvements in patient quality of life. Historical screening process shifts the healthcare paradigm from late-stage reactive treatment to proactive early intervention. A longitudinal study conducted by Van Der Steeg et al. (2010) has uncovered that patients that falls under the false-positive diagnostic result has a lower quality of life and feelings of anxiety. In addition, given how society has transitioned to modern standards with access to higher resolution digital screenings and robust technology, patients will have greater expectations of receiving the optimal care for early-stage diagnosis. On the operational level, the standard of modern mammography is expected to mitigate diagnostic delays and reduce radiologist fatigue. Consecutively, this integration will benefit patients in terms of faster turnaround times to establish treatment plans, potentially raising survival rates while conserving their psychological state.

Breast cancer screening process with mammography

The purpose of mammography is not just to replace traditional screening techniques with a specialised imaging technique, but to screen for underlying abnormalities, known as tumours, embedded in the breast organ that cannot be seen with naked eyes. People also opt for mammography during health screening to lower the risk of dying from late-stage breast cancer. The general screening process involves the patient being placed in an x-ray machine that can capture images of the breast tissue from different angles, these images will be interpreted and analysed by radiologists.

Radiologists are responsible for extracting pathology results based on the mammogram captured. Some indicators of breast cancer are (Uscher, 2026):

- **Calcifications** – small deposits of white specks of calcium in the breast tissues, can be marker of underlying cancer development or benign based on their shape, size and how they are distributed
- **Lumps and masses** – most of them are non-cancerous but screening is recommended for clarity
- **Breast density and asymmetry** – breasts architectural distortion can change over time due to lifestyle, but abnormal changes can be considered as an indicator
- **Malignant growth** – cancerous cells growth in the breast tissue that has the potential to spread to other parts of the body

The human breast is a heterogenous organ exhibiting diverse attributes that maps to different clinical diagnoses that manifest as distinct medical conditions (e.g. number of calcifications, presence of lumps, varying breast tissue densities). With reference to a

paper written by Guirguis et al. (2021), there is spectrum of benign conditions that may or may not be cancerous. Malignant cases also can be manifested in different variations or symptoms that only through the mammogram scans can be detected.

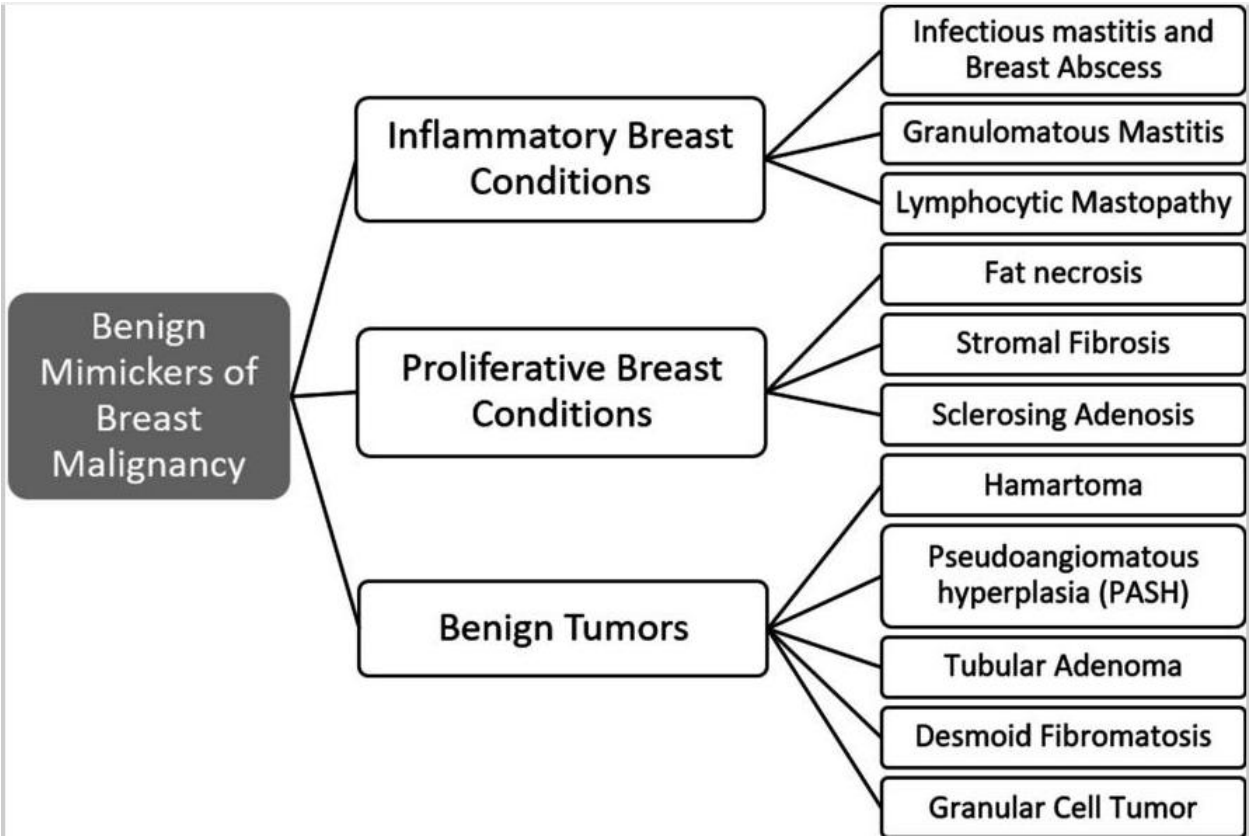
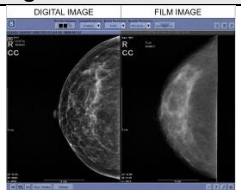
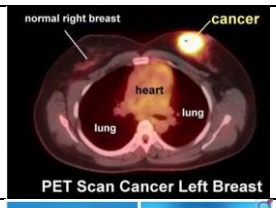
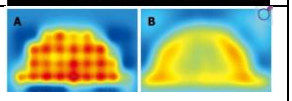


Figure 6 - Categories of benign breast diseases that can mimic breast cancer (Guirguis et al., 2021)

Breast cancer screening technique

The search for the most effective technique for breast cancer detection did not end with mammography. There are limitations with mammography that make other techniques seem more favourable. According to a comparative review of breast cancer imaging techniques by Wang L (2024), mammography is accessible and has better overall image quality; it is limited by dense tissue and radiation exposure. Alternatives like MRI have higher sensitivity that can capture minor details but are costly to operate, and diffused optical tomography is a non-invasive technique but poses problems during reconstruction.

Techniques	Advantages	Disadvantages	Reference	Images
Mammogram	• Relatively high sensitivity • Fast performance and accessible • Generally effective for detecting cancer on dense breast tissues • Provides both 2D and 3D	• Exposure to radiation • Does not have the best image contrast (compared to MRI), might affect sensitivity	(Abeelh & AbuAbeileh, 2024) (Chen et al., 2021) (Jaglan et al., 2019) (Verma et al., 2026) (Wang L, 2024)	
Magnetic Resonance Imaging (MRI)	• Better quality image, making it highly effective for detecting cancer on dense breast tissues • Highest recorded sensitivity	• Expensive to operate • Longer processing time • Low specificity • Not standardised	(Abeelh & AbuAbeileh, 2024) (Chen et al., 2021) (Jaglan et al., 2019) (Verma et al., 2026) (Wang L, 2024)	
Ultrasound (sonography)	• Not exposed to ionising radiation (no radiation) • Effective for distinguishing solid lumps and cystic masses	• Low specificity • Dependent on the technician's skills • Cannot detect microcalcifications	(Abeelh & AbuAbeileh, 2024) (Chen et al., 2021) (Jaglan et al., 2019) (Verma et al., 2026) (Wang L, 2024)	
PET-CT	• Second highest recorded sensitivity • Used for staging cancer and evaluate treatment plan	• Low specificity • Not for early screening of breast cancer • Poor image resolution	(Verma et al., 2026) (Jaglan et al., 2019)	
Diffused optical tomography	• Not exposed to ionising radiation (no radiation) • Unlike mammography, there is no need for compressions on the breasts	• Limited spatial resolution • Cannot detect microcalcifications	(Lee, 2010) (Altoe et al., 2019) (Wang L, 2024)	

Despite the trade-offs, statistically mammography remains as the undisputed benchmark for breast cancer detection. Mammography's clinical benefits outweigh its technical limitations and have proven to drastically reduce breast cancer mortality rates. The limitations, however, can establish a baseline that demands augmentation to refine the current diagnostic precision and clinical practices. Other techniques listed in the table above are still essential for providing secondary screening pathways to manage complex or

ambiguous exceptions. For example, in certain cases, MRI works as an adjunct to mammography as it can detect and evaluate blood flow in breast masses and its risk prediction models are suited for women at high risk for invasive breast cancer. Ultrasound (sonography) is controversial due to its disadvantages, however, in certain cases the ultrasound probe had proven it can detect more cancers than mammography alone (Drukteinis et al., 2013).

Deep Learning integration in mammography

In 1998, the FDA approved computer-aided detection (CAD) systems to be integrated into mammography practices to assist radiologists with the image interpretation and decision-making process by detecting the thicker layers of lesions and analysing the image data for abnormalities (Giger et al., 2008). A study was conducted to evaluate the effectiveness of implementing traditional CAD systems, showing a decreased diagnostic specificity from 90.2% to 87.2% and a rise in biopsy rate of 19.7%. This suggests that despite the substantial increase in biopsies, the actual improvement in breast cancer detection was statistically insignificant (Fenton et al., 2007)

Artificial Intelligence (AI) was introduced to modern CAD systems as an effort to mitigate the limits of traditional systems, whereby the goal is to have them simplify the mammogram analysis process and uncover properties that derive relationships or properties that indicates signs of cancer.

DL is a subset of Machine Learning (ML) that is also a form of AI that can process complex data using neural network architecture that replicates the structure of a human brain to help make meaningful insights. According to Wang L (2024) research, there are several DL architectures that can be used applied to mammography and CAD systems, for detecting breast cancer, classifying the cancer types, segmentation and restoring mammogram images.

One of DL's prominent models is the Convolutional Neural Network (CNN), which is recognised for its pattern recognition capabilities using convolutional layers to extract and analyse features of the images input. It can process and analyse layers of complex image data and patterns, made up of different shapes, edges, colour variations and spatial frequencies. Modern mammography has already introduced CNNs to assist radiologists with the breast cancer classification process. With reference to Wang L (2024), the Nottingham Histological Grading (NGH) system has established three factors that determines the breast cancer scores on top of detecting the primary features of cancer presence:

- **Tubule formation** – one of the critical assessment factors for understanding the level of cancer
- **Nuclear pleomorphism** – defines the boundaries and geometric features of the cancer cell
- **Mitotic count** – an indicator that determines how aggressive the tumour is

The integration of AI or any subsets of AI has shown considerable potential in reducing human error and diagnostic inconsistencies tremendously, providing better more consistent and reliable screening across diverse healthcare settings (Nasir et al., 2025). Nonetheless, regulatory compliance will continue to shape the trajectory of AI adoption in mammography.

Challenges and limitations of CNN

Despite the promising performance of CNN models in breast cancer detection, CNN models are considered problematic for various reasons (Mohammad, 2025). One of which is the learning approach adopted is designed for single task (i.e. classification or segmentation) without leveraging the synergy between the two simultaneously. For example, if CNN identifies where the breast lesion is (segmentation), it should be able to know what type of lesion it is (classification).

CNN also lack the ability to capture global context. Because CNN models are designed to analyse images on small, localised area, there could be challenges with its effectiveness in capture broader contextual information across the entire mammogram.

The third challenge points to the nature of CNN models. Like “black box”, the models decision-making process can pose problems due to the lack of transparency and subjective human domain experts’ judgement or validation. This create the discussion on how accurate and reliable the model’s reasoning is, raising concerns around accountability and patient safety.

Challenges and limitations of Machine Learning/Deep Learning in breast cancer detection

While ML and DL have been revolutionised in modern mammography, there are still conceivable complications to be noted on. First challenge is limited good quality data for training, acquiring medical imaging data ethically is a tedious process, especially for rare

cases that requires long-term observation. Maintaining a balanced dataset is equally critical to ensure a reliable performance of the model (Wang L, 2024).

The second challenge is the regulatory and ethical perspective of using artificial intelligence in clinical practices, where patients' apprehensions regarding data privacy and other safety concerns remain an absolute priority.

The third challenge concerns the potential disruption to hospital standard operating procedures that are already using ML/DL in their CAD/ CAE systems. Since the long-term goal is to have ML/DL assist radiologists in the diagnostic process, any faults in the model will obstruct the standard clinical workflow and affect the turnaround times.

Past projects on breast cancer detection using Machine Learning/Deep Learning

In Wang L's research papers (2024), table three consist of a comprehensive list of DL methods in mammography for breast lesion segmentation. While these are not suitable for our project where our primary goal is having a classification model, certain approach like the Conditional Generative Adversarial Networks (cGAN) through the methods of data augmentation.

Another DL approach used for segmentation is Residual Attention U-Net (RU-Net) can be applied for classification. A project has adopted this model for segmenting breast mass detected in mammogram images and using it to classify the diagnosis and attained satisfactory results with high accuracy (Abdelhafiz et al., 2019). RU-Net was also experimented on outside of breast cancer research, one being abdominal Computed Tomography (CT) scans to segment liver tumours by performing deep feature extractions and precise segmentation (Huang & Huang, 2025) and classifying lung cancer detected in CT scans (Kurmari et al., 2024).

Various CNN frameworks like DenseNet, ResNet and VGGNet have been extensively studied for breast tumour detection (Wang L, 2024). The results from the research done have shown promising outcomes, whereby CNN's false positive rates are lower than radiologist's. CNN's performance can be highly accurate and fast, but its power is dependent on the amount data were trained on. Smaller datasets may not be sufficient for CNN to be at its peak performance, sometimes not being able to detect all forms of breast cancer.

Alternate advanced CNN methods are introduced to challenge the limitations and improve the accuracy of predictions. For example, transfer learning uses pre-trained DL models to

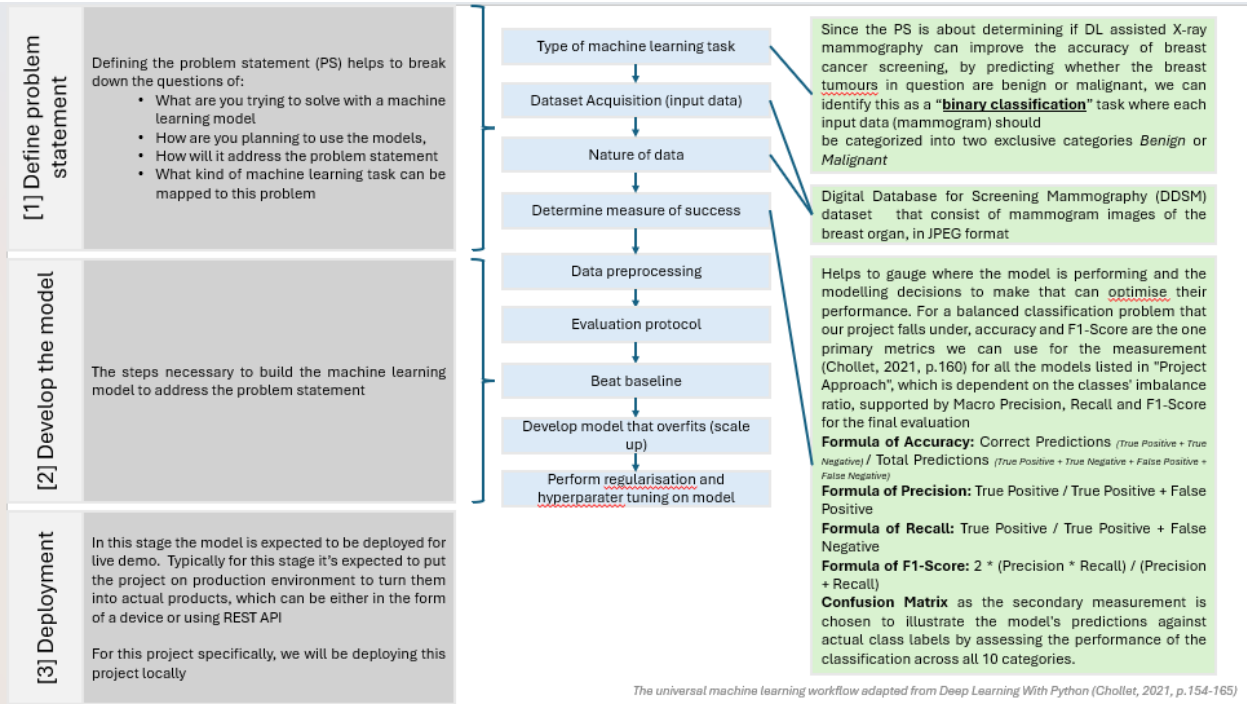
train on small datasets while ensemble learning method involves a combination of multiple DL models using approaches such as stacking, boosting and bagging, and there are attention-based methods that utilise attention mechanisms to focus on critical features of images (Wang L, 2024). Data augmentation also serves as a critical mechanism to handle the data scarcity problem and introduce variety into the datasets, by expanding the variance of the training data it significantly enhances the capability of the model (Salama & Aly, 2021).

Project name	Objective	Model	Results (performance metrics)	Source
Residual DL System for Mass Segmentation and Classification	For segmenting mass lesions in mammogram images, and classify them into benign and malignant	RU-Net	Accuracy: 0.98 Dice coefficient index (DI): 0.98 Intersection over union (IOU): 0.94	Abdelhafiz et al. (2019)
Liver Tumour Classification and Segmentation from CT Images	For classifying and segmenting liver tumours based on abdominal CT scans	RU-Net (ResNet50-UNET)	Accuracy: 96.7% Dice similarity coefficient DSC (liver segmentation: 98.2% DSC (tumour segmentation): 94.6%	Huang and Huang (2025)
Optimized Residual Attention U-Net-Based Lung Cancer Detection	For detecting patterns and anomalies in CT scans to detect signs of lung cancer	RU-Net	AUC: 98.65% F1-score: 87.98% Specificity: 97.22%.	Kurmari et al. (2024)
Breast Mass Tumour Classification using Deep Learning	For classifying pre-segmented mammogram mass tumours as benign or malignant using transfer learning	InceptionV3, ResNet50	**Inception** Accuracy: 0.796 Precision: 0.754 Recall: 0.891 **ResNet50** Accuracy: 0.857 Precision: 0.857	Abdel Rahman et al. (2020)

			Recall: 0.873	
CNN Based Breast Cancer Risk Stratification Using a Mammographic Dataset	Leveraged on a novel CNN derived pixel-wise breast cancer risk model	-	Accuracy: 72%	Ha et al., (2018)

3 Design (1563 words)

This section comprises a high-level workplan and design of the project. The design of the workflow is heavily based on the universal ML workflow from the *Deep Learning with Python* textbook. We have defined the problem statement in the 'Introduction' section, the subsequent sub-sections will discuss more on the design process for building the models that address the classification problem.



The universal machine learning workflow adapted from Deep Learning With Python (Chollet, 2021, p.154-165)

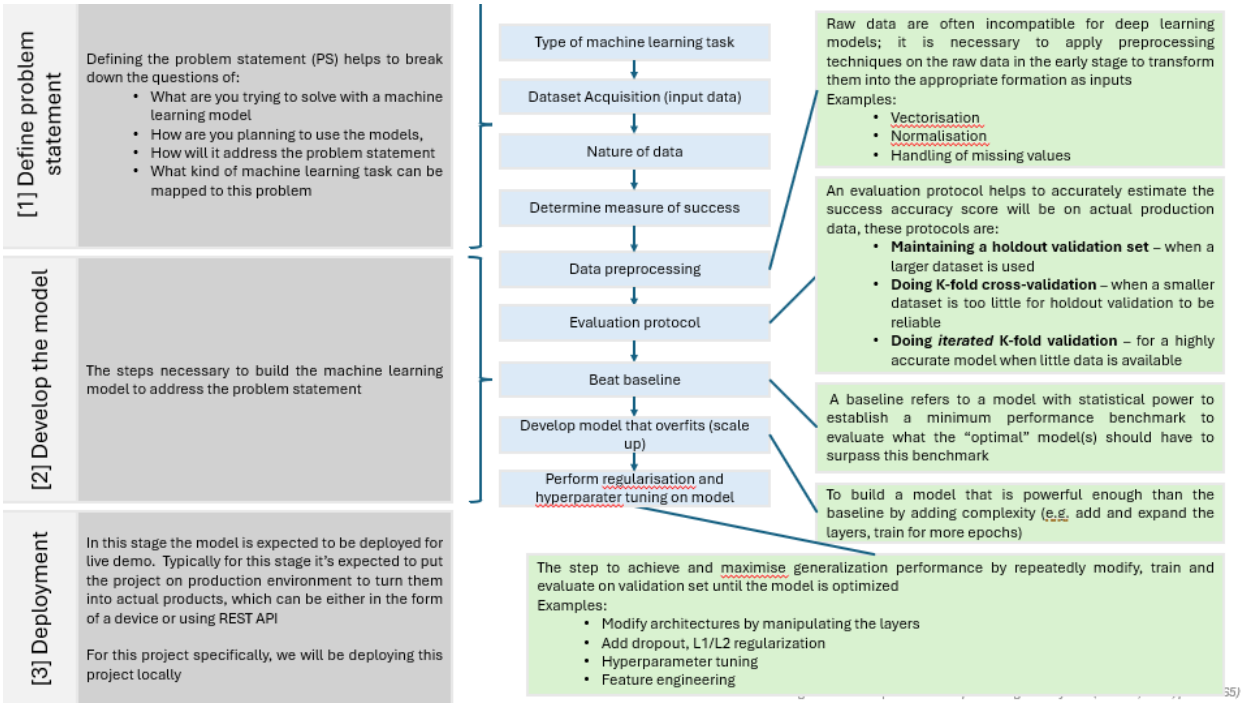


Figure 7 - The universal machine learning workflow adapted from Deep Learning with Python (Chollet, 2021, p.154-165) (Appendix B)

Project Timeline

A Gantt chart is used to map out the overview of the project schedule and deliverables. Since this is the preliminary stage, additional deliverables will be added subsequently as the project progresses.

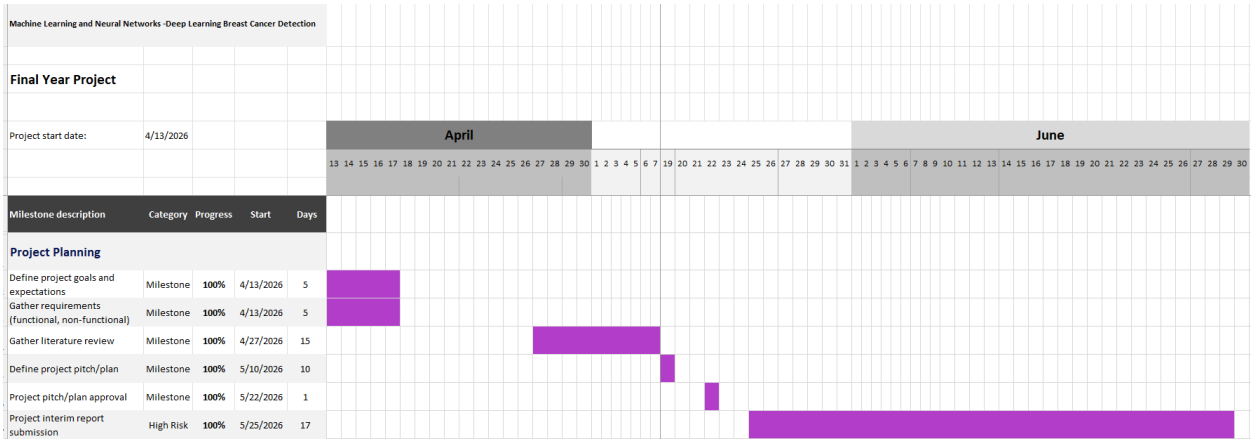


Figure 8 - Gantt Chart (Appendix A)

Code Pipeline

The following diagram depicts the flowchart of the project's code pipeline, along with a table of description on the key elements.

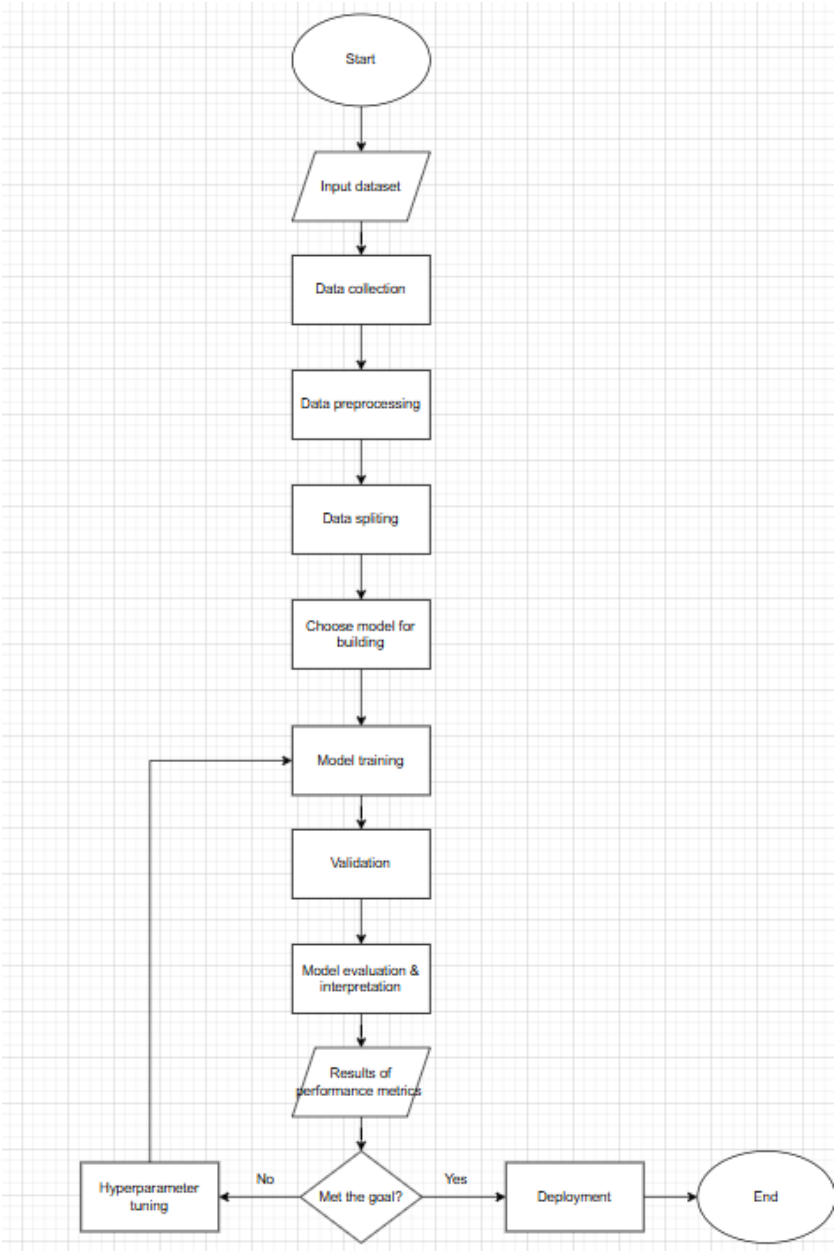


Figure 9 - Flowchart of the project pipeline (Appendix B)

The pipeline is expected to process the mammogram image as the input dataset, which will be reviewed and prepared for the model. We have to standard all the image data to be in consistent format before the model can start learning. Modifications like flipping the

image, fine tuning the image contrast will be expected to make the model more adaptable to variations. The dataset will be divided by train, validation and test sets. The CNN model is expected extract and analyse features of the breast image. Finally, the model is ready to classify the patient with a benign or malignant tumour. The model performance is measured using performance metrics, and a training curve will be plotted to visualise the improvement progress.

Dataset Description

The dataset used for this project is the Curated Breast Imaging Subset of DDSM (CBIS-DDSM) from Kaggle, which provides mammogram images in JPEG format, that is a standardised and widely supported format for DL preprocessing pipelines.

The original database is from sourced the Cancer Imaging Archive (Sawyer-Lee, et al., 2016) containing 2620 scanned mammogram films with a mix of verified “normal”, “benign” and “malignant” diagnosis. According to the source (Lee et al., 2017), the original DDSM images came in non-standard compression format that is incompatible with modern computers, making direct use impractical. Their manuscript detailed the work to curate and covert the database into a standardised CBIS-DDSM format specifically intended for use in CAD and CAE research, making it accessible for DL related projects.

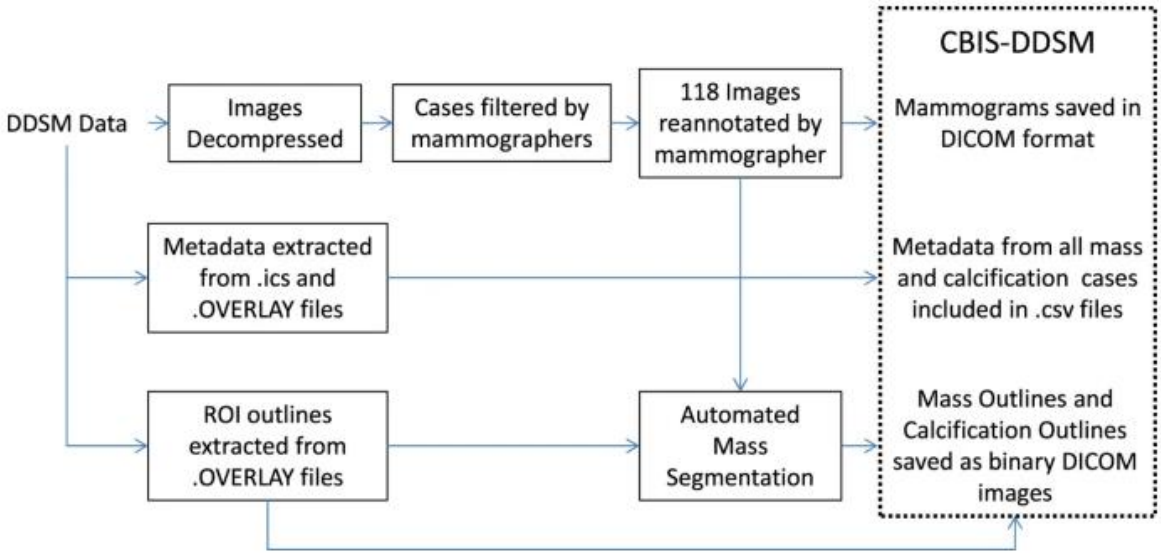
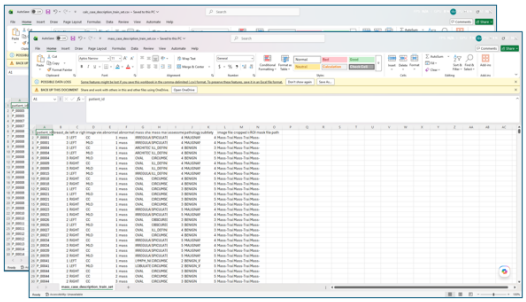


Figure 10 - Flow diagram of preparation of CBIS-DDSM (Lee et al., 2017)

The following table contains additional information about the CSV files that comes with the dataset:

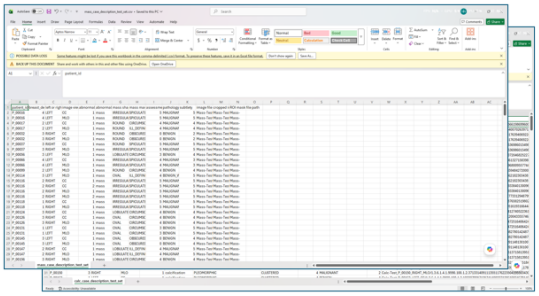
Filename	Description
Dicom_info.csv	For mapping the original DDSM in DICOM format to its correspondence in JPEG format stored locally
Calc_case_description_train/test.csv	Contains pathology labels for calcification cases (e.g. cancer type, ROI shape, breast density, etc)
Mass_case_description_train/test.csv	Contains pathology labels for mass cases (e.g. cancer type, mass shape, mass margin, breast density, etc)
Meta.csv	Additional metadata on the patients

The final submission will use *calc_case_description_train/test.csv* and *mass_case_description_train/test.csv* as the datasets for train and test set respectively. They contain relevant metadata on the pathology label, abnormalities and features that can determine the breast cancer type. Because the current datasets are separated by mass and calcification cases, additional preprocessing steps will be required to combine the training and test files into single dataset, ensuring the model is trained and evaluated across both lesion types.



calc_case_description_train.csv &
mass_case_description_train.csv

calc_case_description_test.csv &
mass_case_description_test.csv



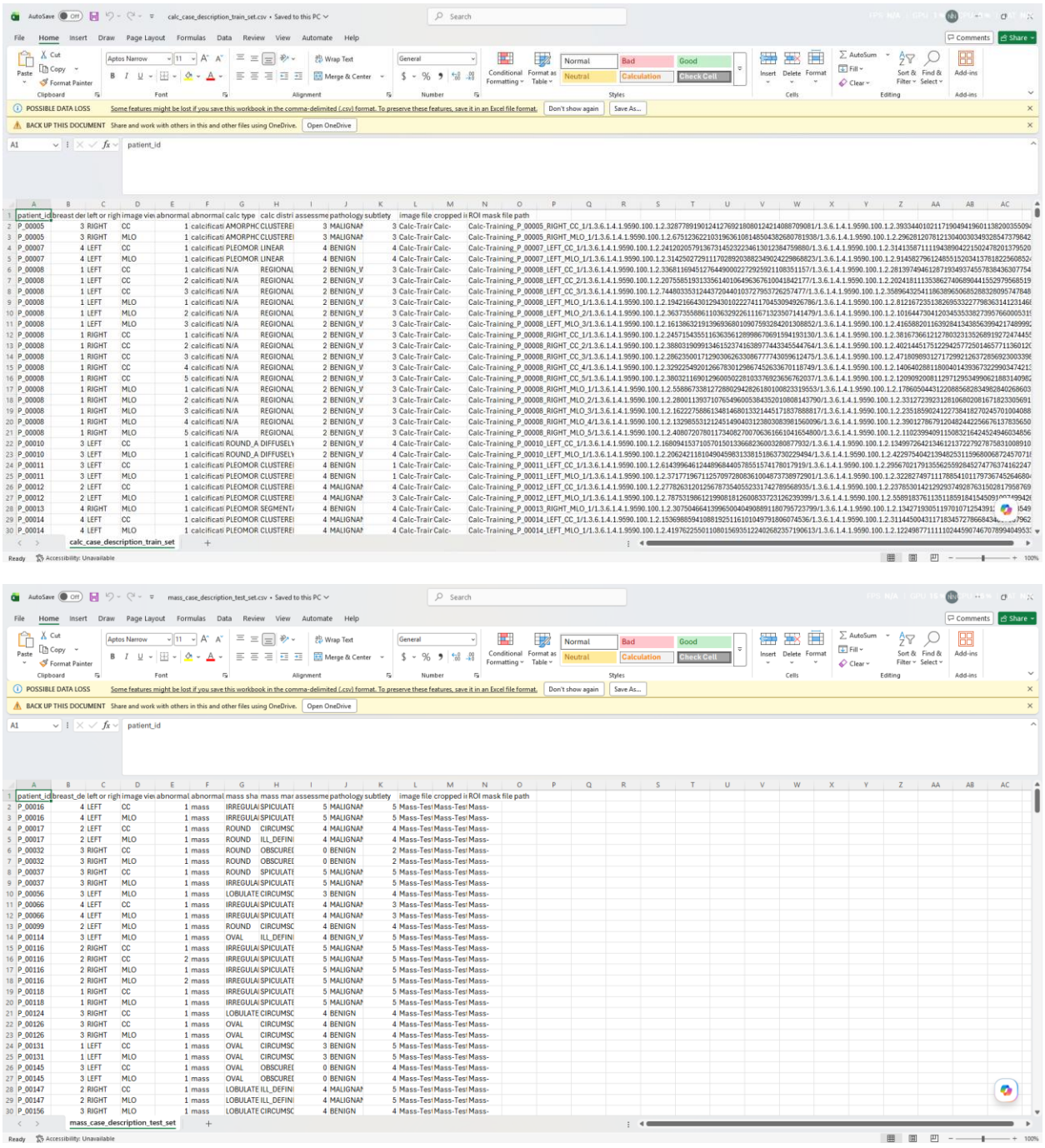


Figure 11 - How the datasets will be processed for final submission (Appendix C)

We will use the mass and calcification “train” datasets for training and their “test” datasets for final evaluation. Details the dataset handling and pre-processing can be found under the “Implementation” section of the report.

Measure Of Success

Defining a measure of success would help to gauge where the model is performing and the modelling decisions to make that can optimise their performance. For a balanced classification problem that our project falls under, accuracy is the primary metrics we can use for the measurement (Chollet, 2021, p.160), also being supported by Loss, AUC, F1-Score, Macro Precision and Recall and for the final evaluation (refer to Appendix D).

Confusion Matrix as the secondary measurement is chosen to illustrate the model's predictions against actual class labels.

CNN Model Architecture

The base CNN model accepts 224 by 224 grayscale images with a single input channel. It contains five convolutional layers that use 3 by 3 kernels and ReLU activation for detecting local patterns that forms the structure of the image.

As the network deepens, the number of filters increase from 32 to 256 allowing the layers to learn simple features and deeper layers to capture more complex patterns.

MaxPooling layers are placed after the first four convolutional layers to reduce the spatial dimensions of the feature maps while retaining important features.

The final feature maps are flattened into a one-dimensional vector and passed to a fully connected layer containing 64 neurons with ReLU activation.

Finally, a single neuron output layer with sigmoid activation produces a probability for binary classification, predicting whether the input belongs to class 0 (benign) or class 1 (malignant).

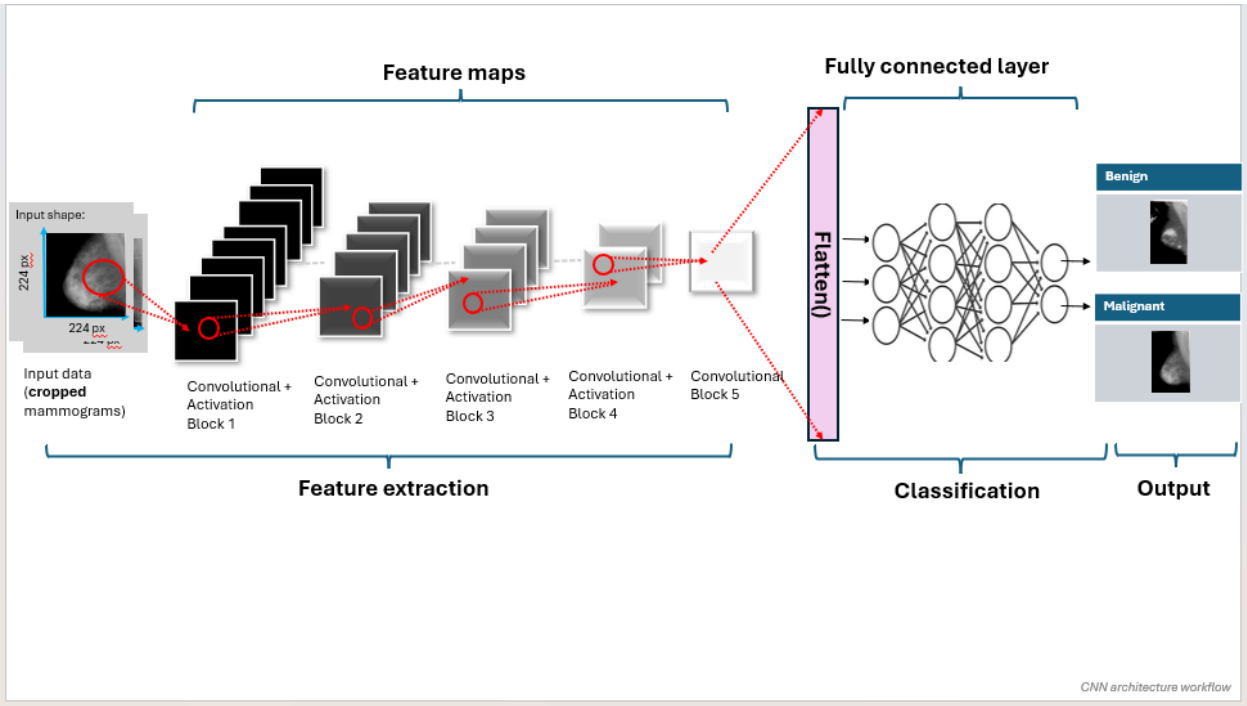


Figure 12 - CNN architecture workflow (Appendix C)

Model: "base_cnn"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_4 (Conv2D)	(None, 10, 10, 256)	590,080
flatten (Flatten)	(None, 25600)	0
dense (Dense)	(None, 64)	1,638,464
dense_1 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

Trainable params: 2,616,449 (9.98 MB)

Non-trainable params: 0 (0.00 B)

Figure 13 – Sample CNN model architecture

The following are the DL models used for this project's case study. Transfer learning was used to reuse features learned by ResNet50 and VGG16 from the large ImageNet dataset. The ResNet50 backbone was frozen to preserve its pre-trained weights, whereas VGG16 was unfrozen to allow its features adapt to our input images. The decision for designing the CNN models can be found in Appendix E.

Custom CNN models	Pre-trained CNN models
Baseline Establish a performance benchmark for how the CNN model is expected to surpass and provide the feedback needed to improve on its performance. To attain the results, we use a simple random classifier to benchmark the minimal performance.	**ResNet50** Known as Residual Networks, ResNet is a pre-trained CNN model that spans across a variety of family groups, representing the different architecture depths (He et al., 2015). With reference to Wang L's (2024) paper, ResNet is a common applied model for breast lesion detection cases using mammography. Given its relevance to similar field of study, we can attempt to experiment it for our project. ResNet50 was chosen with weights pre-trained on the ImageNet dataset, allowing previously learned features to be transferred to our classification task (transfer learning).
Base CNN model We build a base CNN model that follows the architecture in *figure. 11* that can outperform the baseline, with the expectation that it will overfit/underfit.	**VGG16** Known as Visual Geometry Group, VGG is also another common pre-trained CNN model, with a family of different architecture depths used for DL-based breast tumour detection (Wang L, 2024). VGG16 was selected because it offers substantial representational depth while containing fewer layers than VGG19, making it a suitable and comparative model for this project given the hardware constraints.

CNN model with augmented data We add another layer that contains the augmented data, to expand the diversity of the training dataset. The augmentation consists of the following operations applied to the image data: • Flipping • Rotations • Image translation • Zoom • Image contrast	
CNN model with Dropout and Regularisation The base model's processing power is not enough to recognise abstract and complex features. We implement dropout and regularisation techniques to address the gaps.	
CNN model with tuned hyperparameters Using *hyperband()* function from KerasTuner to explore the range of values for the hyperparameter that can optimise the model.	

Evaluation Protocol

An evaluation protocol is for measuring the generalisation power of the ML model as the generalisation power represents how well the model can interpolate on new unseen data (Chollet, 2021, p.133 & 152). A well-performing model must balance the trade-offs of being optimised and generalisation. To perform evaluation, we have adopted a stratified holdout-validation protocol to preserve the class distribution across the data partitions. We split the dataset into, and:

- **70% training** – for learning features and patterns of the input image data
- **10% validation (of the train set)** – for monitoring the model's performance, determine the number of epochs and selecting the best hyperparameters
- **20% test set** – contains the unseen data excluded from training for final evaluation of the model

Model performance was evaluated using performance metrics from the 'Measure of Success' sub-section. Training and validation performance was monitored during model development, with the validation results used to set an expectation on how the model will generalise to unseen data, and for determining the most suitable number of training epochs and hyperparameters to optimise the model's performance. The final model was evaluated on the unseen test set. The same test partition and performance metrics were used, allowing a fair comparison of their generalisation performance. The final model's success was determined by whether it:

1. Surpass baseline's accuracy score with the unseen test set particularly in accuracy metric
2. Demonstrate learning and generalisation capabilities, with a higher validation accuracy and lower validation loss scores for each model iteration
3. Maintain a balanced classification performance across both classes by achieving high numbers of true-positive and true-negative predictions while minimising false-positive and false-negative predictions

Each model iteration was evaluated using the same validation protocol and metrics to determine the effect of the optimising step. We use an iterative model design approach to progressively add new layers and hyperparameters, to do a comparison of the techniques that has contributed to the improvement in generalisation and whether each model provided a measurable improvement over the baseline.

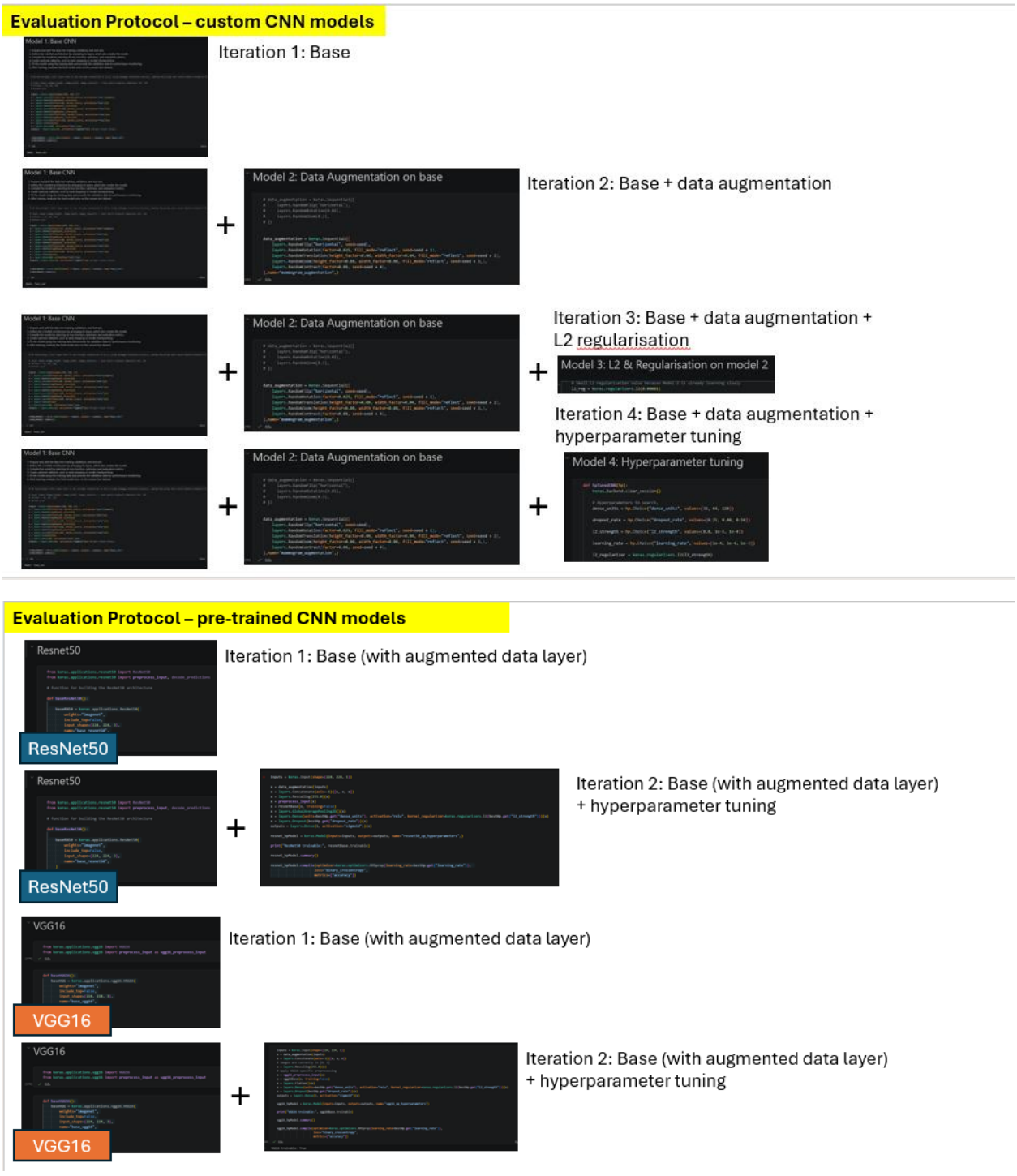


Figure 14 - Iterative model design (Appendix C)

4 Implementation (1128 words)

This section details the steps to create the CNN models for the project. The model is created with reference to the Tensorflow, Keras documentation and *Deep Learning with Python* textbook. The project code can be found in the Appendix H.

The GitHub repository to the project: https://github.com/hngnyng/UOL-FYP-2026

Importing and loading datasets

We start off with importing the necessary libraries and load the datasets.

```
import os
import re
import PIL
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.utils import class_weight
from sklearn.preprocessing import LabelEncoder
from skimage.transform import resize
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
import keras_tuner as kt

# Dataset's folder name in local machine
localPath = r"CBISDDSM-AWSAF"
# Search for the file containing the path to the dataset (images)
csvDicomPath = os.path.join(localPath, "csv", "dicom_info.csv")
csvMassTrainPath = os.path.join(localPath, "csv", "mass_case_description_train_set.csv")
csvCalcTrainPath = os.path.join(localPath, "csv", "calc_case_description_train_set.csv")
csvMassTestPath = os.path.join(localPath, "csv", "mass_case_description_test_set.csv")
csvCalcTestPath = os.path.join(localPath, "csv", "calc_case_description_test_set.csv")
imagePath = os.path.join(localPath, "jpeg")
```

Explorative Data Analysis (EDA)

We perform a simple EDA on the dataset to get an overview of the type of images available in the dataset.

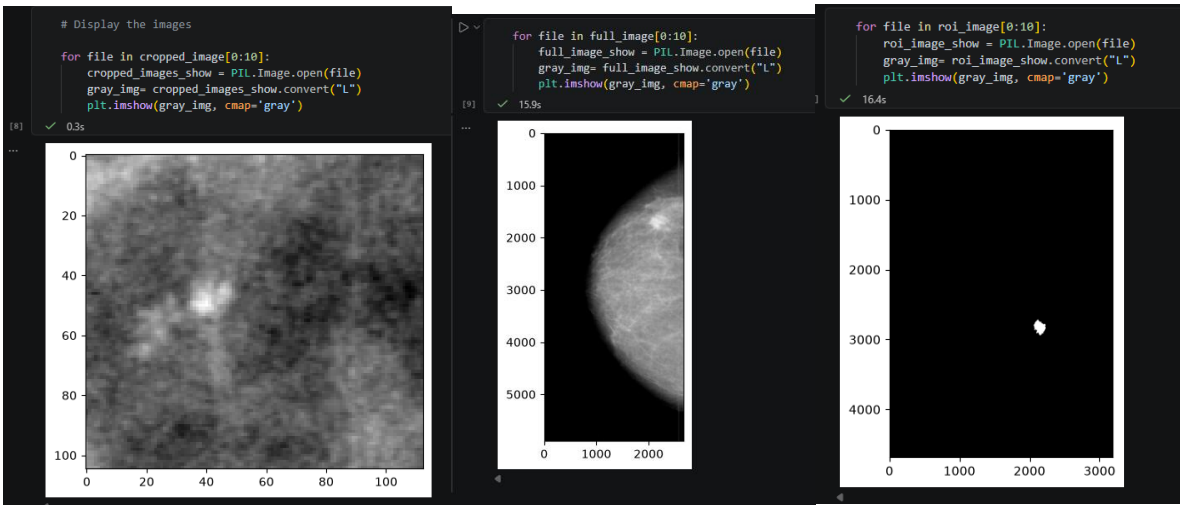


Figure 15 - From left to right: cropped, full, ROI mammogram images

Dicom_info.csv (df_dicom) contains the direct local directory paths of the downloaded images. Based on the distribution of the images, cropped mammogram images has the greatest number of samples available. Therefore, they were selected for model training.

```
images = pd.DataFrame(df_dicom['SeriesDescription'].value_counts())
images = images.reset_index()
images = images.rename(columns={'SeriesDescription': 'Image Types', 'index':'SeriesDescription'})
images
```

0.0s

	Image Types	count
0	cropped images	3567
1	ROI mask images	3247
2	full mammogram images	2857

Figure 16 - Image sample distribution

Resolving issues with mass/calcification train datasets

The local directory paths of the mammogram images in the raw mass and calcification datasets (df_mass, df_calc) do not directly correspond to usable local file paths. To access the images locally, we must extract the paths from *df_dicom['image_path']* dataset to populate in *image_file_path*, *cropped_image_file_ path*, *ROI_mask_file_path* of df_mass and df_calc.

Each image in df_dicom is associated with a SeriesInstanceUID (a unique folder identifier) and a SeriesDescription (tag for indicating image is a full mammogram, a cropped lesion patch, or an ROI mask). We can use *df_dicom['SeriesInstanceUID']* and *df_dicom['SeriesDescription']* to look up each row of df_mass and df_calc by for the corresponding UID labelled in the faulty path string and the appropriate image type. Where multiple candidate files matched a given UID, image type pair, following the dataset's naming convention of *{Mass|Calc}-{Training|Test}_{patient_id}_ {breast side}_{view}_ [{abnormality ID}],* we reconstruct each case's expected identifier and select the candidate who's stored PatientID matched exactly. Rows for which no valid match could be found were left unresolved rather than assigned an unverified path and were subsequently dropped from model training. The same method was applied to the mass/calc test sets (df_testMass, df_testCalc).

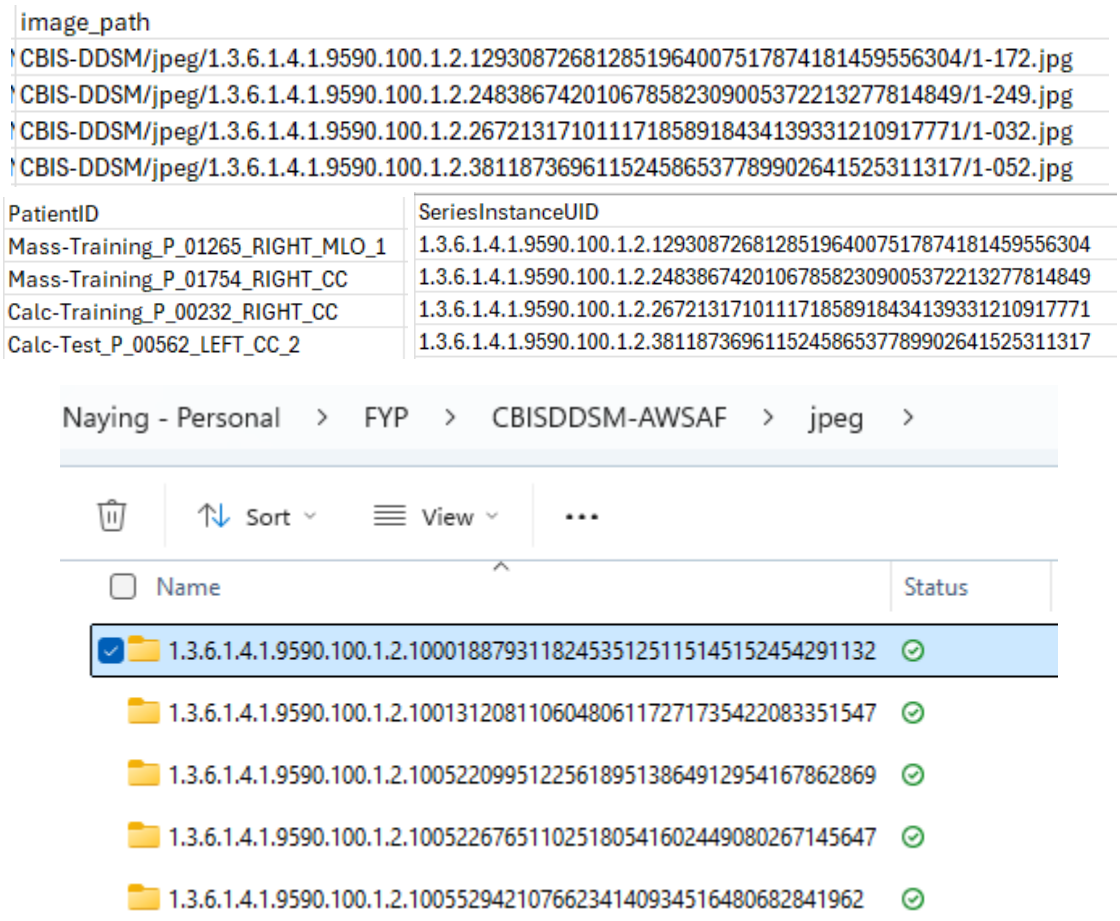


Figure 17 - Metadata from dicom_info.csv needed to access images in local directory

image file path
Mass-Training_P_00001_LEFT_CC/1.3.6.1.4.1.9590.100.1.2.422112722213189649807611434612228974994/1.3.6.1.4.1.9590.100.1.2.342386194811267636608694132590482924515/000000.dcm
Mass-Training_P_00001_LEFT_MLO/1.3.6.1.4.1.9590.100.1.2.319478999311971442426185353560182990988/1.3.6.1.4.1.9590.100.1.2.359308329312397897125630708681441180834/000000.dcm
Mass-Training_P_00004_LEFT_CC/1.3.6.1.4.1.9590.100.1.2.347107867812656628709864319310977895697/1.3.6.1.4.1.9590.100.1.2.891800462110225318343526314836693465 40/000000.dcm
Mass-Training_P_00004_LEFT_MLO/1.3.6.1.4.1.9590.100.1.2.272600286511817402806912403581910920939/1.3.6.1.4.1.9590.100.1.2.295360926313492745441868049270168300162/000000.dcm
cropped image file path
Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.9590.100.1.2.108268213011361124203859148071588939106/1.3.6.1.4.1.9590.100.1.2.296736403313792599626368780122205399650/000000.dcm
Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9590.100.1.2.188473874511440575807446266233629582526/1.3.6.1.4.1.9590.100.1.2.227955274711225756835838775062793186053/000000.dcm
Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.9590.100.1.2.414182170112396175925115449620455230167/1.3.6.1.4.1.9590.100.1.2.429120414011832984817094399141838850375/000000.dcm
Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9590.100.1.2.342549219112801262604988012930063078769/1.3.6.1.4.1.9590.100.1.2.115134232113001553100559896703407510515/000000.dcm

ROI mask file path
Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.9590.100.1.2.108268213011361124203859148071588939106/1.3.6.1.4.1.9590.100.1.2.296736403313792599626368780122205399650/000001.dcm
Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9590.100.1.2.188473874511440575807446266233629582526/1.3.6.1.4.1.9590.100.1.2.227955274711225756835838775062793186053/000001.dcm
Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.9590.100.1.2.414182170112396175925115449620455230167/1.3.6.1.4.1.9590.100.1.2.429120414011832984817094399141838850375/000001.dcm
Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9590.100.1.2.342549219112801262604988012930063078769/1.3.6.1.4.1.9590.100.1.2.115134232113001553100559896703407510515/000001.dcm

Figure 18 - Incorrect image paths in mass/calc datasets

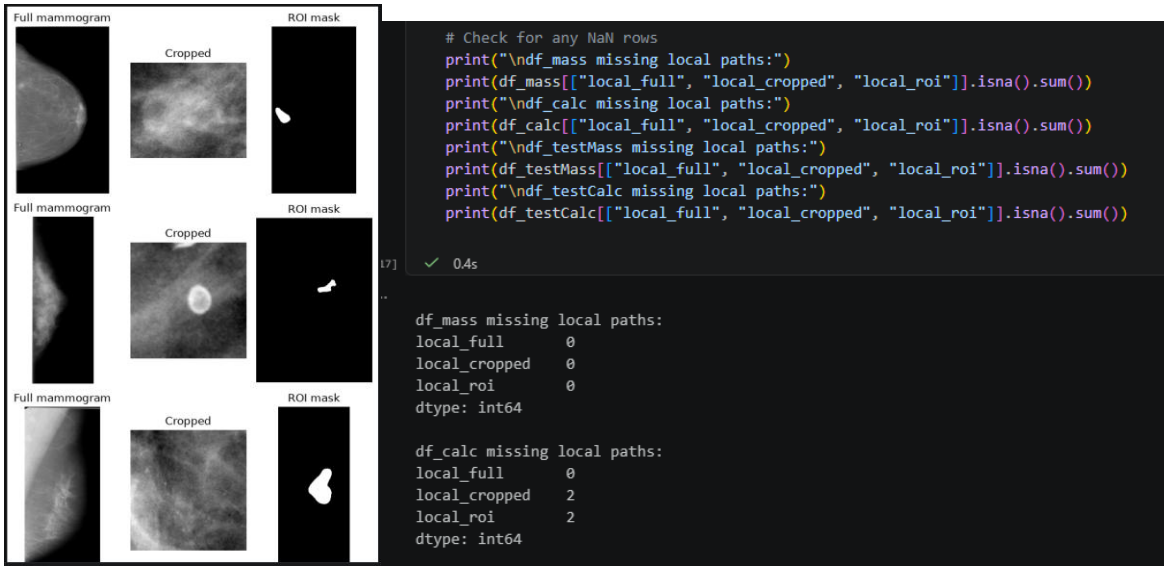


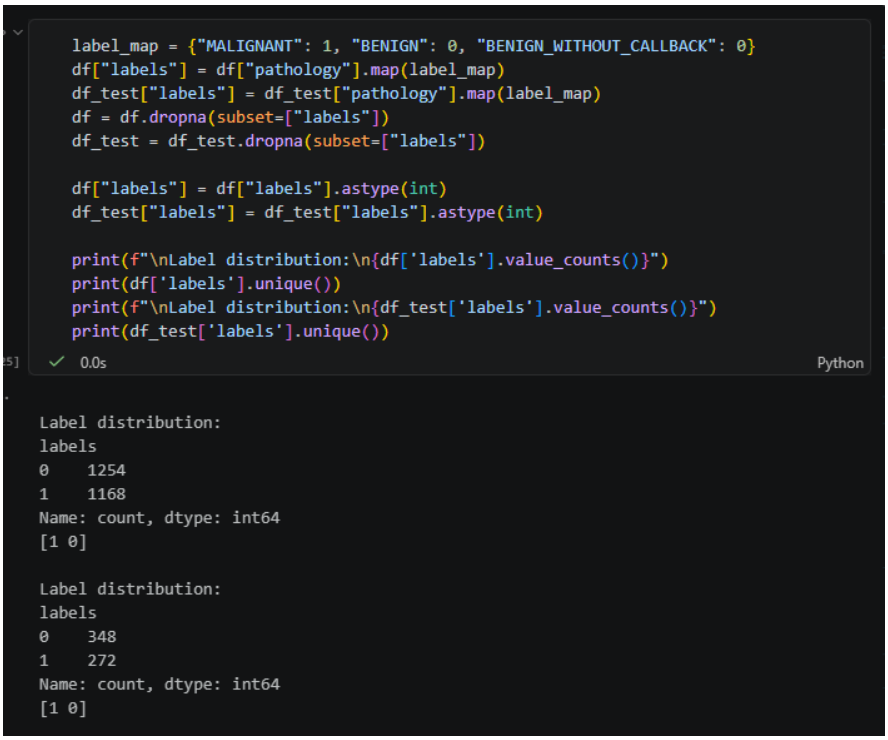
Figure 19 - Results after mapping

Binary label encoding

In the original datasets, there are three labels for pathology: BENIGN_WITHOUT_CALLBACK, BENIGN and MALIGNANT. We convert the pathology categories into binary labels, assigning 1 to malignant cases and 0 to both benign

categories. The labels are then converted to integers, and their distributions are printed to confirm that both datasets contain only classes 0 and 1.

The output shows 1254 benign and 1168 malignant *training* samples, compared with 348 benign and 272 malignant *test* samples.



```
label_map = {"MALIGNANT": 1, "BENIGN": 0, "BENIGN_WITHOUT_CALLBACK": 0}
df["labels"] = df["pathology"].map(label_map)
df_test["labels"] = df_test["pathology"].map(label_map)
df = df.dropna(subset=["labels"])
df_test = df_test.dropna(subset=["labels"])

df["labels"] = df["labels"].astype(int)
df_test["labels"] = df_test["labels"].astype(int)

print(f"\nLabel distribution:\n{df['labels'].value_counts()}")
print(df['labels'].unique())
print(f"\nLabel distribution:\n{df_test['labels'].value_counts()}")
print(df_test['labels'].unique())
```

0.0s Python

```
Label distribution:
labels
0    1254
1    1168
Name: count, dtype: int64
[1 0]

Label distribution:
labels
0    348
1    272
Name: count, dtype: int64
[1 0]
```

One-hot encoding

Before splitting the dataset, the images and their corresponding labels were pre-processed to prepare them for training and evaluation using the CNN models. One-hot encoding converts each categorical label into a numerical vector that distinguishes the two classes without implying an ordered relationship between them.

Each image is opened using PIL and converted to grayscale with *.convert("L")*. The image is then converted into a NumPy array so that it can be processed numerically. and resized to the dimensions specified by *imageSize* (224 * 224)

LabelEncoder assigns an integer to each class (e.g. 0, 1). This integer encoded class labels were converted into a "two-class" one-hot format using *to_categorical*. Benign encoded as 0 is represented as [1, 0], while malignant encoded as 1 is represented as [0, 1]. The resulting one-hot encoded label array was converted to the float32 data type.

This creates a consistent machine-readable format for subsequent data processing, having the processed images stored in the NumPy array X, while their corresponding one-hot encoded labels stored in the array y.

```
images, labels = [], []

for _, row in df.iterrows():
    path, label = row['local_cropped'], row['labels']

    # Open and decode the image file
    # grayscale = single channel, shape (H, W)
    image = PIL.Image.open(path).convert("L")
    image = np.array(image)

    # Resize to 224 with imageSize
    # shape: (224, 224), values already scaled to [0, 1]
    # using from skimage.transform import resize
    image = resize(image, imageSize, anti_aliasing=True).astype("float32")
    # Convert uint8 [0, 255] pixels to float32 [0, 1]. normalisation step
    # image = np.asarray(image, dtype=np.float32,) / 255.0
    images.append(image)
    labels.append(label)


# One hot encoding
label_encoder = LabelEncoder()
encodedLabels = label_encoder.fit_transform(labels)

X = np.stack(images).astype("float32")
# Converting one-hot labels to binary labels
y = to_categorical(encodedLabels, num_classes=2,).astype("float32")



# no. of samples, w, h
print("X shape:", X.shape)
# no. of samples,
print("y shape:", y.shape)
print (y)
```

✓ 35.4s

```
X shape: (2422, 224, 224)
y shape: (2422, 2)
[[0. 1.]
 [0. 1.]
 [1. 0.]
 ...
 [0. 1.]
 [1. 0.]
```

Split train, validation, test data

The following are the details to the spliced datasets:

- 70% training – for learning features and patterns of the input image data
    - X_main_train represents *train + validation set combined – 1937 samples*
    - X_train represents *train set only – 1743 samples*
- 10% validation (of the train set) – for monitoring the model's performance, determine the number of epochs and selecting the best hyperparameters – *194 samples*

- 20% test set – contains the unseen data excluded from training for final evaluation of the model
    - X_test represents the *20% test set* derived from *df – 485 samples*
    - df_test is a separate test dataset

```
# 70% train set, 20% test set
# 10% of train --> validation set



# X/y_train_main refers to the train set before splitting between validation-test sets
X_train_main, X_test, y_train_main, y_test = train_test_split(X, y, test_size = 0.20,
                                                              stratify = y,
                                                              shuffle = True,
                                                              random_state = 42)

print(f"Training set, no. of samples (70%): {len(X_train_main):}")
print(f"Test set, no. of samples (20%): {len(X_test):}")
print(type(X_train_main[0]))
print(X_train_main[0])
```

```
[33]  ✓ 0.1s

Training set, no. of samples (70%): 1937
Test set, no. of samples (20%): 485
<class 'numpy.ndarray'>
[[0.30210096 0.31966004 0.34554905 ... 0.4135789  0.43761992 0.49230504]
 [0.2916529  0.3123802  0.32163835 ... 0.41479522 0.43387744 0.47810543]
 [0.2800924  0.30629128 0.31127125 ... 0.4258494  0.4402351  0.47251385]
 ...
 [0.30747184 0.2527439  0.31930694 ... 0.18686444 0.19528395 0.21225975]
 [0.2850722  0.26318434 0.32157457 ... 0.1831803  0.19777015 0.1947337 ]
 [0.26442227 0.2904626  0.31472406 ... 0.18089943 0.20235406 0.1667474 ]]
```

Figure 20 - X_train_main and X_test split

```
Get validation set (10% of X_train_main)

X_train, X_val, y_train, y_val = train_test_split(X_train_main, y_train_main,
                                                  test_size = 0.10,
                                                  stratify = y_train_main.argmax(axis = 1),
                                                  shuffle = True,
                                                  random_state = 42)

print(f"Training set: {X_train.shape[0]}")
print(f"Validation set: {X_val.shape[0]}")
print(f"Test set: {X_test.shape[0]}")
✓ 0.0s

Training set: 1743
Validation set: 194
Test set: 485
```

Figure 21 - X_train and X_val split

Pixel normalisation

Here we conduct validation checks to ensure the train, validation, test sets were pre-processed consistently. Here we scale the image pixel values, which will eventually be the input features X, to the range [0, 1].

This helps the models to train more stably, converge faster by providing a consistent input scale, while preventing non-standardised or large pixel values from producing unstable weight updates.

```
print(f"X_train range: [{X_train.min():.3f}, {X_train.max():.3f}]")
print(f"X_val range: [{X_val.min():.3f}, {X_val.max():.3f}]")
print(f"X_test range: [{X_test.min():.3f}, {X_test.max():.3f}]")

assert X_train.dtype == np.float32
assert X_val.dtype == np.float32
assert X_test.dtype == np.float32

assert 0.0 <= X_train.min() and X_train.max() <= 1.0
assert 0.0 <= X_val.min() and X_val.max() <= 1.0
assert 0.0 <= X_test.min() and X_test.max() <= 1.0
✓ 0.2s

X_train range: [0.000, 1.000]
X_val range: [0.000, 1.000]
X_test range: [0.000, 1.000]
```

Establish Baseline's accuracy

This formula calculates the majority class baseline accuracy. Because *y_train_bin* contains only 0 and 1, its mean is equal to the proportion of samples labelled 1.

For example, if 52% of the training samples belong to class 0, a classifier that predicts class 0 for every image would obtain a baseline accuracy of 52%.

```
baseline = max(np.mean(y_train_bin), 1 - np.mean(y_train_bin))
print(f"Baseline accuracy: {baseline:.3%}")
```

✓ 0.0s

```
Baseline accuracy: 51.807%
```

Converting input into tensors

A CNN model (convnet) takes in and operates on tensor objects as input. To ensure our image arrays are formatted to a rank 3 tensor with dimensions of: (height, width, channels), we restore the channel dimension with *np.expand_dims()*, producing the output of (1743, 224, 224, 1) (no. of samples for training, width, height, grayscale channel).

The one-hot encoded labels were converted into binary values using `np.argmax()`, where [1, 0] became class 0 and [0, 1] became class 1. This produced one binary label for each image.

```
# Add the grayscale channel only if it is missing.
if X_train.ndim == 3:
    X_train = X_train[..., np.newaxis]

if X_val.ndim == 3:
    X_val = X_val[..., np.newaxis]

if X_test.ndim == 3:
    X_test = X_test[..., np.newaxis]

# Convert one-hot vectors into binary 0/1 labels.
y_train_bin = np.argmax(y_train, axis=1,).astype("int32")
y_val_bin = np.argmax(y_val, axis=1).astype("int32")
y_test_bin = np.argmax(y_test, axis=1).astype("int32")

print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("X_test:", X_test.shape)

print("y_train:", y_train_bin.shape)
print("First five labels:", y_train_bin[:5])

assert X_train.shape[1:] == (224, 224, 1)
assert X_val.shape[1:] == (224, 224, 1)
assert X_test.shape[1:] == (224, 224, 1)

assert len(X_train) == len(y_train_bin)
assert len(X_val) == len(y_val_bin)
assert len(X_test) == len(y_test_bin)
```

[38] ✓ 0.0s

```
X_train: (1743, 224, 224, 1)
X_val: (194, 224, 224, 1)
X_test: (485, 224, 224, 1)
y_train: (1743,)
First five labels: [0 1 1 1 1]
```

Building the custom models

1. Defining the convnet architecture

That includes constructing the layers that make up the convoluted network.

```
# No Rescaling(1./255) layer here it was already normalized to [0,1] using skimage.transform.resize()

# input shape (image_height, image_width, image_channels) -> must match original dimension 224, 224
# filters = 32, 64, 128
# kernel size

inputs = keras.Input(shape=(224, 224, 1))
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation="relu",)(x)
outputs = layers.Dense(1, activation="sigmoid")(x) #single output binary


cnnBaseModel = keras.Model(inputs = inputs, outputs = outputs, name="base_cnn")
cnnBaseModel.summary()
✓ 0.0s

Model: "base_cnn"
```

Figure 22 - Base CNN

For model 2 (figure 23), we add a layer that contains the augmented data to the base model (model 1).

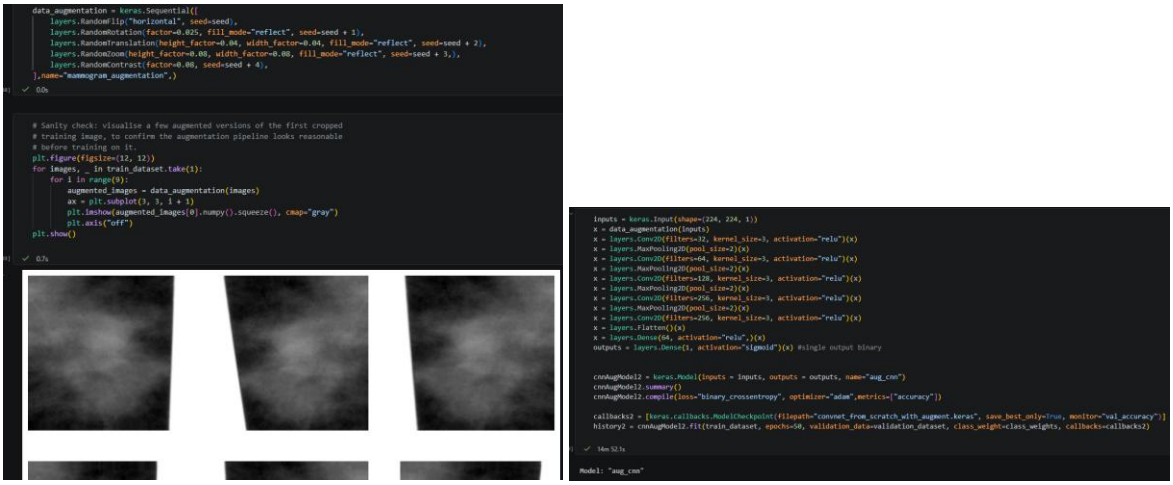


Figure 23 - Base + data augmentation

```
# Small L2 regularisation value because Model 2 is already learning slowly
l2_reg = keras.regularizers.l2(0.00001)

inputs = keras.Input(shape=(224, 224, 1))
x = data_augmentation(inputs)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu", kernel_regularizer=l2_reg)(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu", kernel_regularizer=l2_reg)(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation="relu",)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
# Apply dropout before the final classifier
# x = layers.Dropout(0.25)(x)

cnnl2RegModel = keras.Model(inputs=inputs, outputs=outputs, name = "l2reg_cnn")
cnnl2RegModel.summary()
cnnl2RegModel.compile(loss="binary_crossentropy", optimizer="adam",metrics=["accuracy"])

callbacks3 = [keras.callbacks.ModelCheckpoint(filepath="convnet_with_dropout_regularisation.keras", monitor="val_accuracy", save_best_only=True)]

#     keras.callbacks.EarlyStopping(
#         monitor="val_accuracy",
#         mode="max",
#         patience=10,
#         restore_best_weights=True
#     )
# ]

history3 = cnnl2RegModel.fit(train_dataset, epochs=50, validation_data=validation_dataset, class_weight=class_weights, callbacks=callbacks3)

✓ 17m 39.2s

Model: "l2reg_cnn"
```

Figure 24 - Base + data augmentation + L2 regularisation

For model 4 (figure 25), KerasTuner was used for tuning hyperparameters and attained the following results:

- Dense layer units – 64 neurons

- Dropout rates – 0.25 rates
- L2 regularisation strength – 0.00001
- Adam learning rate – 0.0001
- Best training duration (epoch) – 8

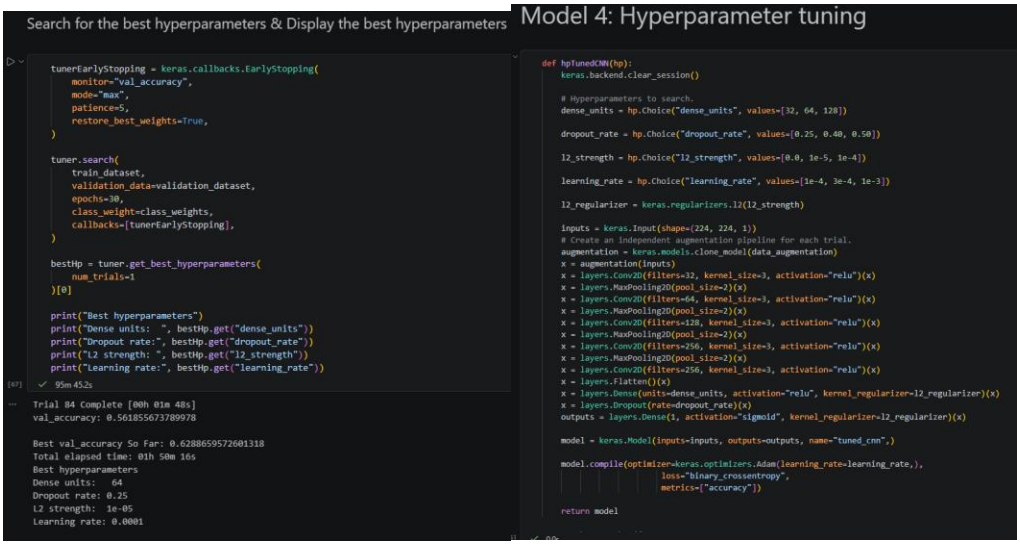


Figure 25 - Hyperparameter tuning results

2. Compile the model by selecting its loss function, optimizer, and evaluation metrics

```
model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate,),
              loss="binary_crossentropy",
              metrics=["accuracy"])
```

Figure 26 - An example of compilation step

3. Create callbacks and class weights

```
cnnBaseModel.compile(loss="binary_crossentropy", optimizer="adam",metrics=["accuracy"])
callbacks = [keras.callbacks.ModelCheckpoint(filepath="convnet_from_scratch.keras", save_best_only=True, monitor="val_accuracy")]
```

```
    weights_array = class_weight.compute_class_weight(
        class_weight="balanced",
        classes=np.unique(y_train_bin),
        y=y_train_bin
    )
    class_weights = dict(enumerate(weights_array))
    print(f"Class weights: {class_weights}")
[40]  ✓ 0.0s
...  Class weights: {0: np.float64(0.9651162790697675), 1: np.float64(1.0375)}
```

Figure 27 - Calculates balanced class weights from the training labels

4. **Fit the model using the training data and provide the validation data for performance monitoring**

We train the longer by implementing epoch 50.

```
history3 = cnnl2RegModel.fit(train_dataset, epochs=50, validation_data=validation_dataset, class_weight=class_weights, callbacks=callbacks3)
```

Figure 28 - Sample of fitting the model

5. **After training, plot the accuracy and loss graph, print the results of the performance metrics, and evaluate the final model once on the unseen test dataset using the confusion matrix**

We create three functions which can be used across models. printPerfMetrics() function to print out the metrics, plotTrainingHistory() function is for generating a training/validation accuracy and loss graph and plotConfusionMatrix() for generating confusion matrix

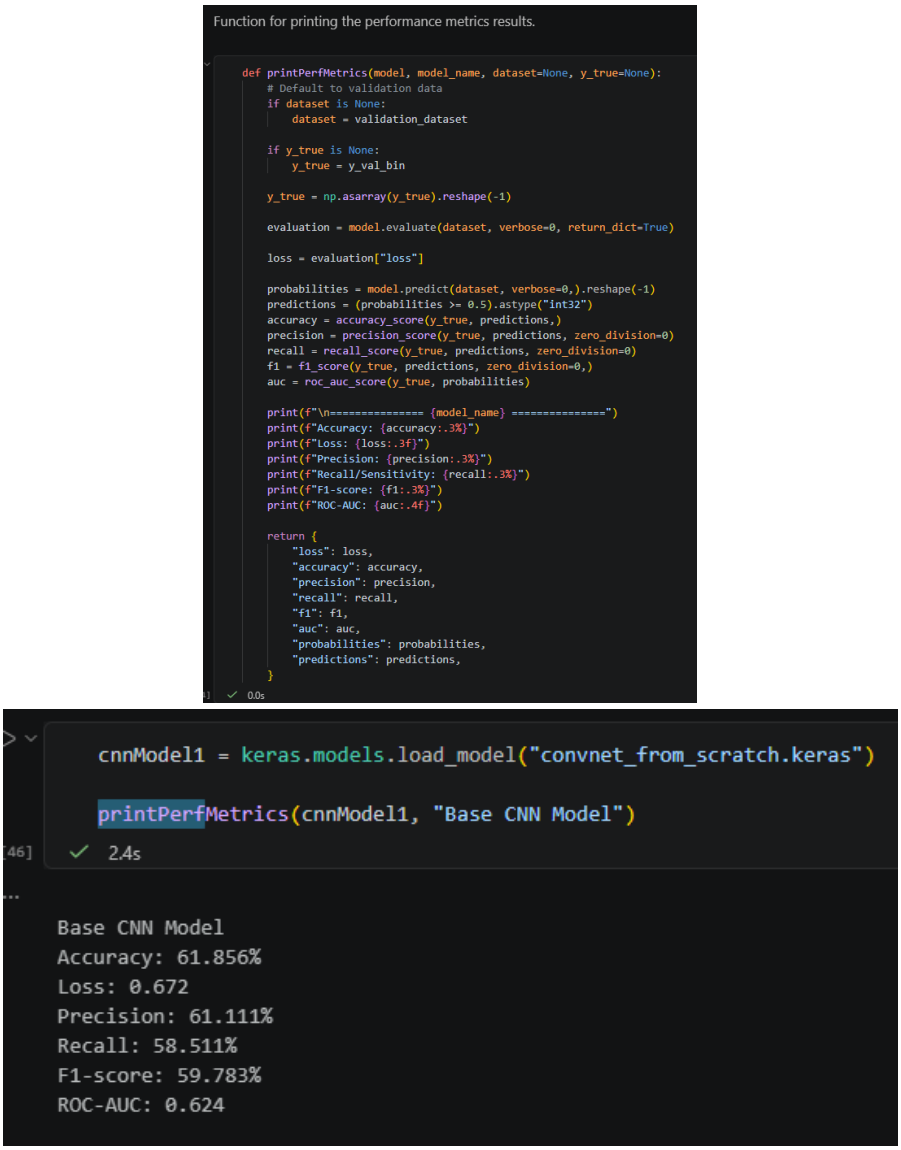


Figure 29 - Function for printing the results and sample of the results

Function for plotting training and validation loss/accuracy graphs.

```python
def plotTrainingHistory(history, title=None):
    fig, axs = plt.subplots(1, 2, figsize=(15, 5))
    epochs_range = range(1, len(history.history["loss"]) + 1)
    title_suffix = f" ({title})" if title else ""

    # Plot 1: Loss
    axs[0].plot(epochs_range, history.history["loss"], "b-", label="Training", linewidth=2,)

    if "val_loss" in history.history:
        axs[0].plot(
            epochs_range,
            history.history["val_loss"],
            "r-",
            label="Validation",
            linewidth=2,
        )

    axs[0].set_title(f"Loss{title_suffix}")
    axs[0].set_xlabel("Epochs")
    axs[0].set_ylabel("Loss")
    axs[0].legend()
    axs[0].grid(alpha=0.3)

    # Plot 2: Accuracy
    axs[1].plot(epochs_range, history.history["accuracy"], "b-", label="Training", linewidth=2,)

    if "val_accuracy" in history.history:
        axs[1].plot(
            epochs_range,
            history.history["val_accuracy"],
            "r-",
            label="Validation",
            linewidth=2,
        )

    axs[1].set_title(f"Accuracy{title_suffix}")
    axs[1].set_xlabel("Epochs")
    axs[1].set_ylabel("Accuracy")
    axs[1].legend()
    axs[1].grid(alpha=0.3)
```

0.0s

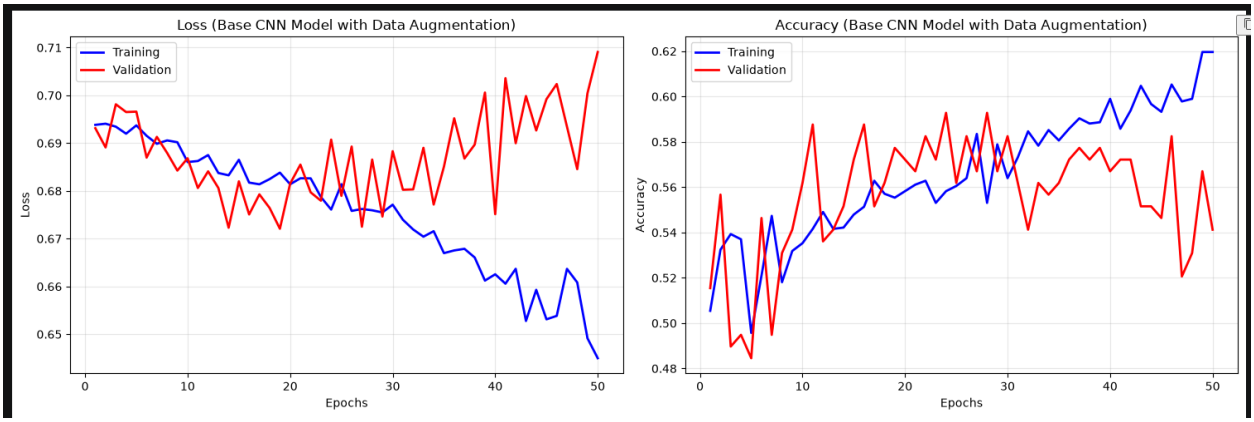


Figure 30 - Function for generating the accuracy/loss graph and sample of the result

Function for generating confusion matrix.

```
def plotConfusionMatrix(yTrue, yPredictions, title="Confusion Matrix",):
    # Ensure both inputs are one-dimensional arrays.
    yTrue = np.asarray(yTrue).reshape(-1)
    yPredictions = np.asarray(yPredictions).reshape(-1)
    assert len(yTrue) == len(yPredictions), ("True labels and predictions must have the same length.")
    matrix = confusion_matrix(yTrue, yPredictions, labels=[0, 1])

    fig, ax = plt.subplots(figsize=(8, 6))
    display = ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=["Benign", "Malignant"])
    display.plot(ax=ax, cmap="Blues", values_format="d", colorbar=False,)

    ax.set_title(title)
    plt.tight_layout()
    plt.show()

    return matrix
```

0.0s

```
dfTestConfusionMatrix = plotConfusionMatrix(y_df_test, dfTestPredictions, title="Confusion Matrix - Final Tuned CNN on df_test")
```

0.0s

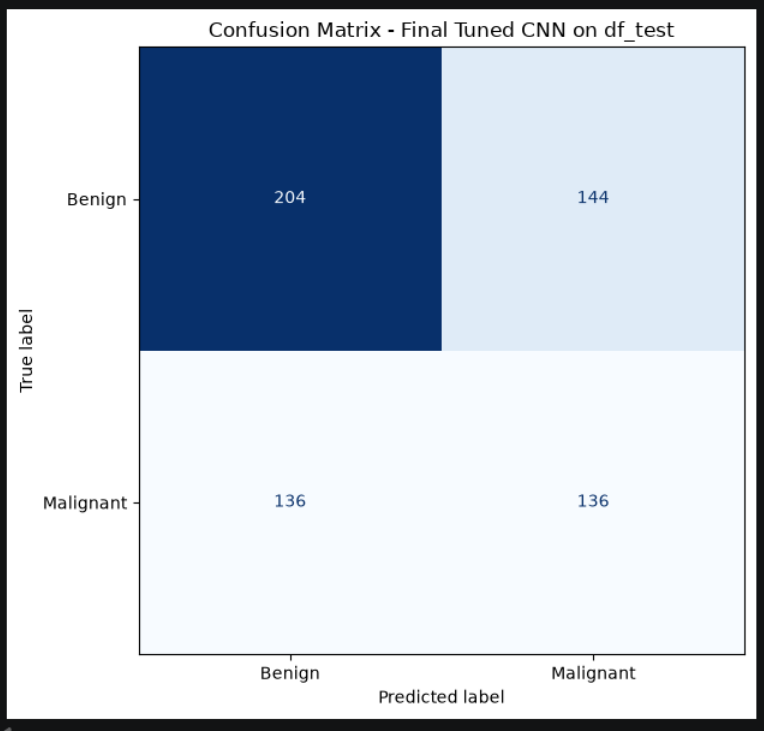


Figure 31 - Function for generating confusion matrix and sample of result using df_test

Building the ResNet50 & VGG16 model

The same steps building the custom CNN models can be applied to ResNet50 and VGG16 models in terms of data pre-processing, model compilation and evaluation. Since these models already learned features from ImageNet and are computationally demanding, we reduce the training duration to epochs 20.

1. Defining the convnet architecture

For both pre-trained architectures, the Keras Applications API was used to create the ImageNet pre-trained ResNet50 and VGG16 backbones. After training, Keras was also used to load the saved model checkpoints containing the architecture and the weights learned by the newly added classification layers.

```
# function for building the ResNet50 architecture

def baseResNet50():

    baseRN50 = keras.applications.ResNet50(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base_resnet50",
    )
    return baseRN50
```

✓ 0.0s

Build the new classifier on top:

```
inputs = keras.Input(shape=(224, 224, 1))
x = data_augmentation(inputs) # include the previous data augmentation layer
# Convert grayscale to three identical channels.
x = layers.Concatenate(axis=-1)([x, x, x])
# ResNet50 preprocess_input expects approximately [0, 255].
x = layers.Rescaling(255.0)(x)
x = preprocess_input(x)

# For extracting features
# training=False is important because ResNet50 contains batch normalisation layers
x = resnetBase(x, training=False)

# Convert the feature maps into a feature vector
x = layers.GlobalAveragePooling2D()(x)
# Regularise the new classifier.
x = layers.Dropout(0.5)(x)
# New binary classifier
outputs = layers.Dense(1, activation="sigmoid")(x)

resnetFeatureModel = keras.Model(inputs=inputs, outputs=outputs, name="resnet50_feature_extract")

print("ResNet50 trainable:", resnetBase.trainable)

resnetFeatureModel.summary()

resnetFeatureModel.compile(optimizer="rmsprop",
                           loss="binary_crossentropy",
                           metrics=["accuracy"])
```

✓ 0.0s

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 1)	0	-
mammogram_augmenta… (Sequential)	(None, 224, 224, 1)	0	input_layer_1[0]…
concatenate (Concatenate)	(None, 224, 224, 3)	0	mammogram_augmen… mammogram_augmen… mammogram_augmen…
rescaling (Rescaling)	(None, 224, 224, 3)	0	concatenate[0][0]
get_item (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_1 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_2 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
stack (Stack)	(None, 224, 224, 3)	0	get_item[0][0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 224, 224, 3)	0	stack[0][0]
base_resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712	add[0][0]
global_average_poo… (GlobalAveragePool…	(None, 2048)	0	base_resnet50[0]…
dropout (Dropout)	(None, 2048)	0	global_average_p…
dense (Dense)	(None, 1)	2,049	dropout[0][0]

Figure 32 - Using Keras to build a ResNet50 base, adding own binary classifier

ResNet50 with optimised hyperparameters

```
inputs = keras.Input(shape=(224, 224, 1))

x = data_augmentation(inputs)
x = layers.Concatenate(axis=-1)([x, x, x])
x = layers.Rescaling(255.0)(x)
x = preprocess_input(x)
x = resnetBase(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(units=bestHp.get("dense_units"), activation="relu", kernel_regularizer=keras.regularizers.l2(bestHp.get("l2_strength")))(x)
x = layers.Dropout(bestHp.get("dropout_rate"))(x)
outputs = layers.Dense(1, activation="sigmoid",)(x)

resnet_hpModel = keras.Model(inputs=inputs, outputs=outputs, name="resnet50_op_hyperparameters",)

print("ResNet50 trainable:", resnetBase.trainable)

resnet_hpModel.summary()

resnet_hpModel.compile(optimizer=keras.optimizers.RMSprop(learning_rate=bestHp.get("learning_rate")),
                       loss="binary_crossentropy",
                       metrics=["accuracy"])
```

```
ResNet50 trainable: False

Model: "resnet50_op_hyperparameters"
```

Figure 33 - ResNet50 with best hyperparameters architecture

Step 7: Building VGG16

```
def baseVGG16():
    baseVGG = keras.applications.vgg16.VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base_vgg16",
    )

    return baseVGG
```

```
[171] ✓ 0.0s
```

```
keras.backend.clear_session()
keras.utils.set_random_seed(seed)

vgg16Base = baseVGG16()

inputs = keras.Input(shape=(224, 224, 1))
x = data_augmentation(inputs)
x = layers.Concatenate(axis=-1)([x, x, x])
# Images are currently in [0, 1]
x = layers.Rescaling(255.0)(x)
# Apply VGG16-specific preprocessing
x = vgg16_preprocess_input(x)
x = vgg16Base(x, training=False)
x = layers.Flatten()(x)
x = layers.Dense(256)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

vgg16Model = keras.Model(inputs=inputs, outputs=outputs, name="vgg16_feature_extract")

print("VGG16 trainable:", vgg16Base.trainable)

vgg16Model.summary()

vgg16Model.compile(optimizer="rmsprop",
                   loss="binary_crossentropy",
                   metrics=["accuracy"])
```

```
✓ 10.5s
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
VGG16 trainable: True
```

Figure 34- Using Keras to build VGG16

2. Compile the model by selecting its loss function, optimizer, and evaluation metrics

```
resnetFeatureModel.compile(optimizer="rmsprop",
                           loss="binary_crossentropy",
                           metrics=["accuracy"])
```

```
✓ 0.0s
```

3. **Create callbacks and class weights**

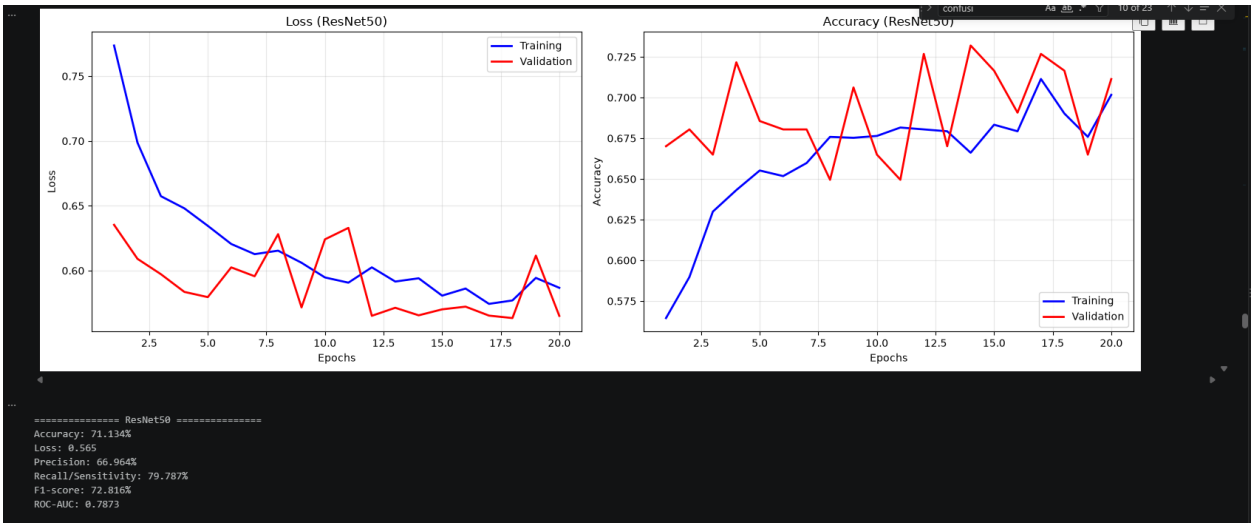
```
vgg16_callbacks= [keras.callbacks.ModelCheckpoint(filepath="vgg16_feature_extract.keras", monitor="val_loss", mode="min", save_best_only=True)]
```

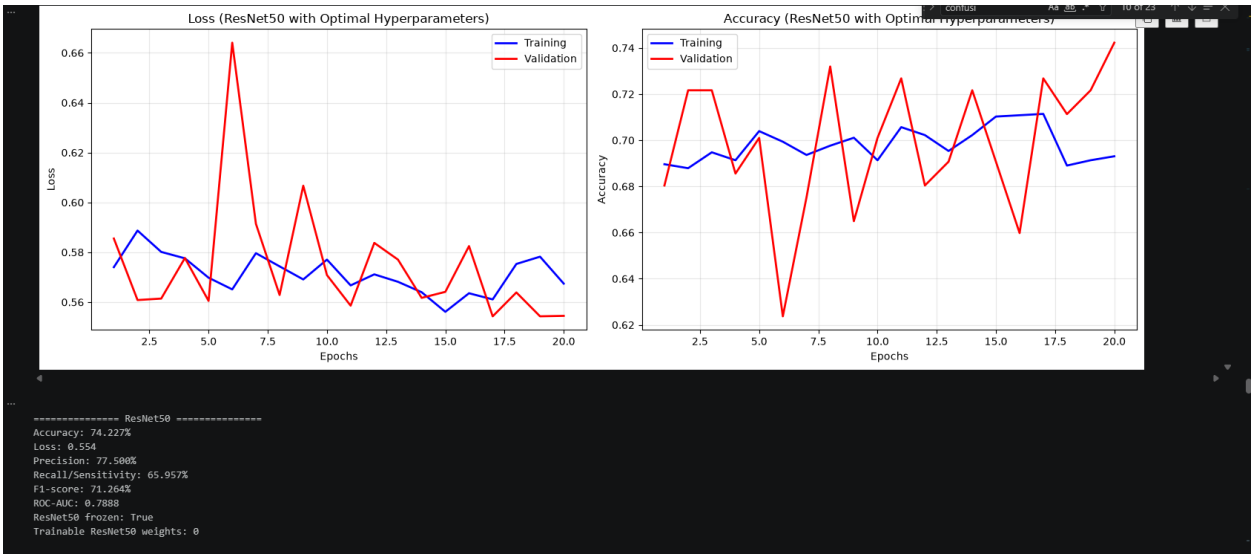
4. **Fit the model using the training data and provide the validation data for performance monitoring**

```
base_resnetCallbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="resnet50_feature_extract.keras",
        monitor="val_loss",
        mode="min",
        save_best_only=True,
    )
]

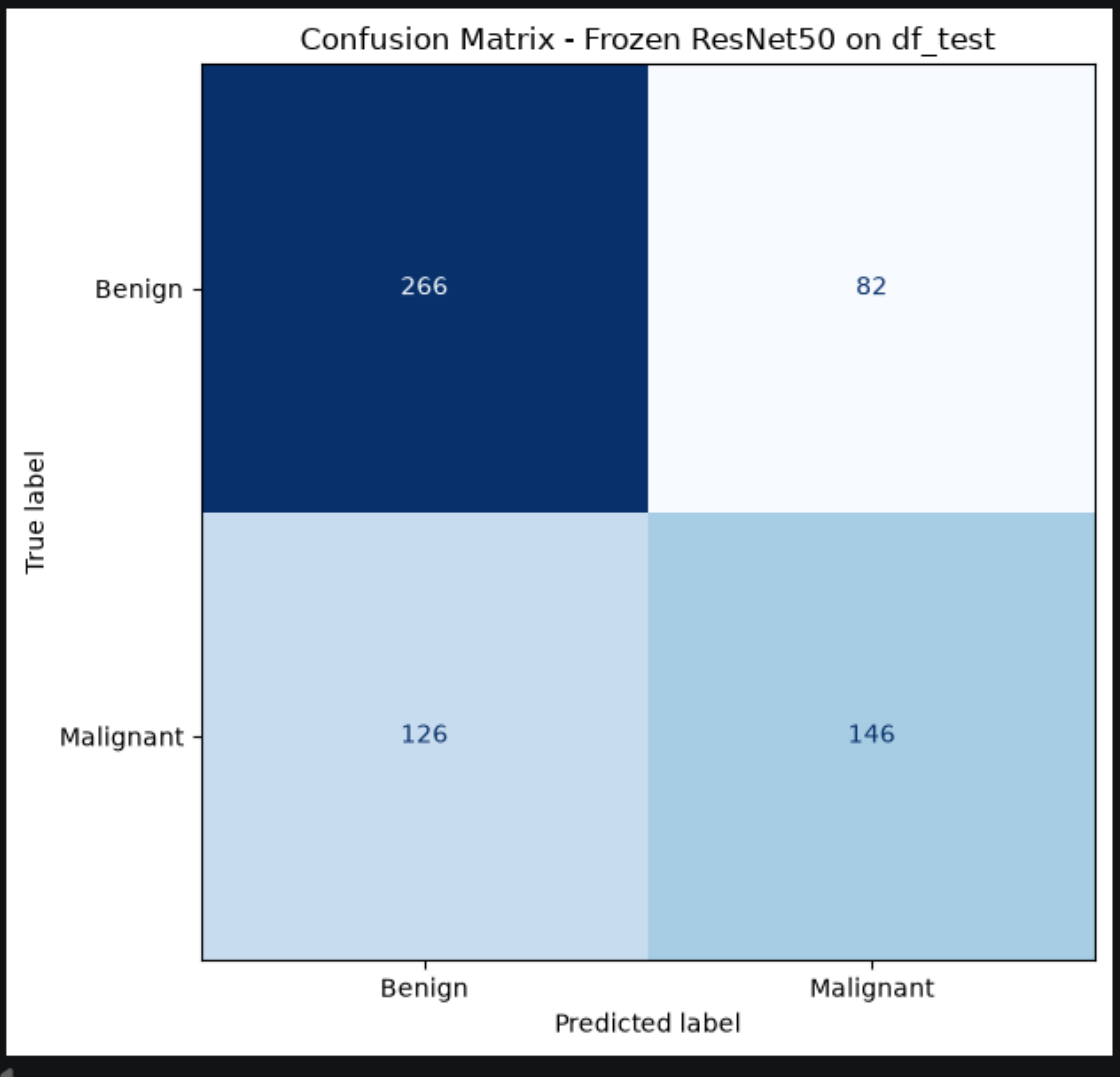
base_resnetHistory = (resnetFeatureModel.fit(resnetTrainDataset,
                                             validation_data=validation_dataset,
                                             epochs=20,
                                             class_weight=class_weights,
                                             callbacks=base_resnetCallbacks))
```

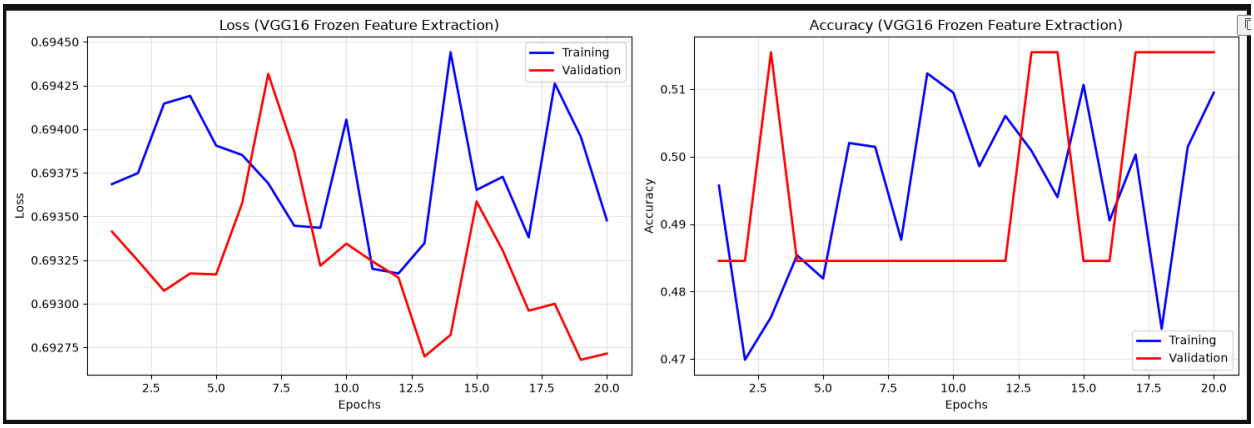
5. **After training, plot the accuracy and loss graph, print the results of the performance metrics, and evaluate the final model once on the unseen test dataset using the confusion matrix**





```
=============== ResNet50 Frozen Feature Extraction - df_test ==============
Accuracy: 66.452%
Loss: 0.611
Precision: 64.035%
Recall/Sensitivity: 53.676%
F1-score: 58.400%
ROC-AUC: 0.7063
```






```
=============== VGG16 Frozen Feature Extraction + Tuned Hyperparameter - df_test ===============
Accuracy: 56.129%
Loss: 0.690
Precision: 0.000%
Recall/Sensitivity: 0.000%
F1-score: 0.000%
ROC-AUC: 0.5000
```

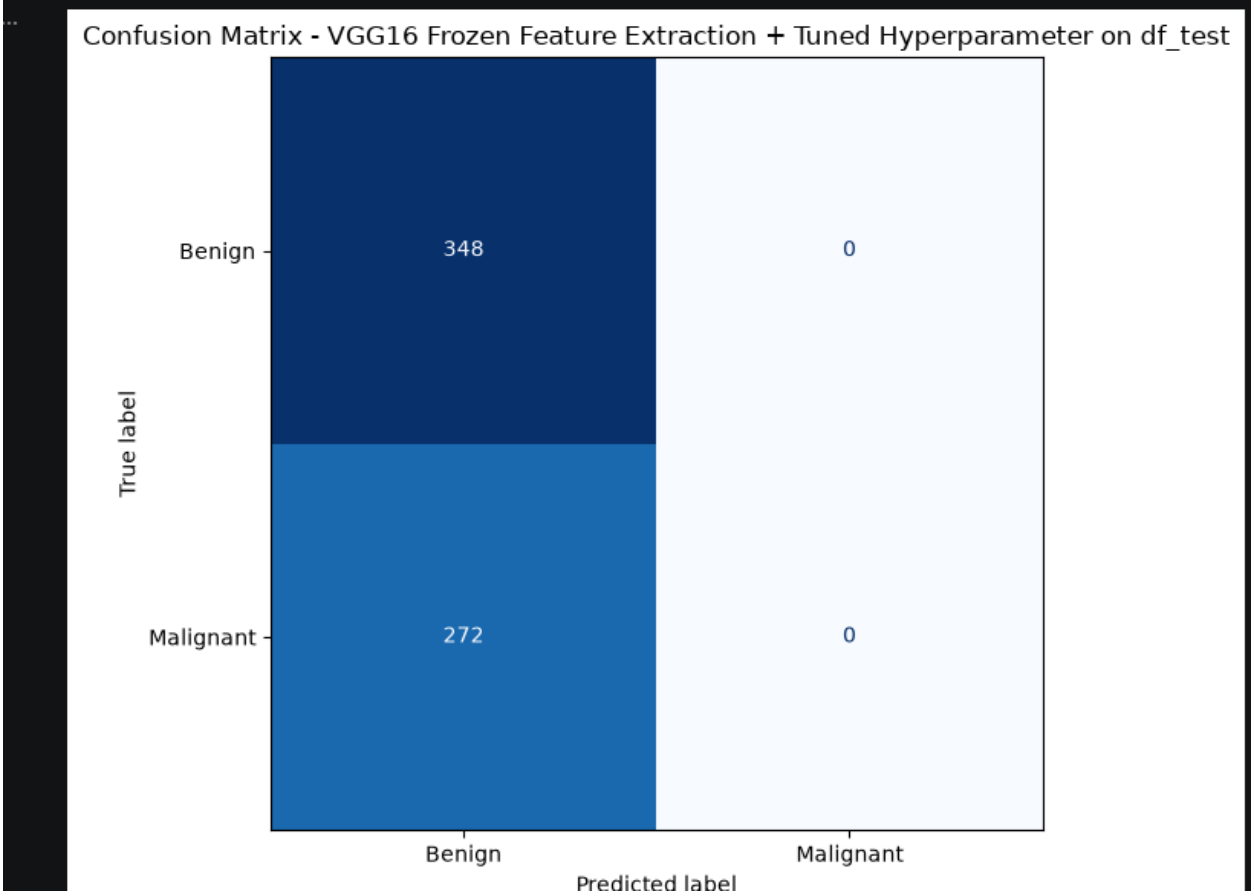


Figure 35 - Sample of the results

5 Evaluation (1071 words)

In this section we review the results from the tests done on the datasets and critically evaluate the project's achievements and recommendations for improvement. The project code and screen captures can be found in the Appendices.

Results

We evaluate each model design iteration using the validation accuracy relative to the baseline, against the measure of success defined in the *Design* section. Screen captures can be found in Appendices.

Custom CNN models

Custom CNN models			
Models	**Metrics (after the last epoch)**	**Metrics (from the best checkpoint)**	**Description**
Baseline	Accuracy: 51.807%		
Base CNN [1] (Appendix F.1)	Validation accuracy: 54.124%	Accuracy: **61.856%** Loss: 0.672 Precision: 61.111% Recall: 58.511% F1-score: 59.783% ROC-AUC: 0.624	By epoch 20, the training curves indicate overfitting, as training accuracy increased and training loss decreased while validation accuracy remained unstable and validation loss increased. The best saved checkpoint achieved 61.856%, exceeding the majority class baseline of 51.807%. However, the F1-score of 59.783% and ROC-AUC of 0.624 indicate only weak generalisation and discrimination between benign and malignant cases.
Base CNN + data augmentation [2] (Appendix F.2)	Validation accuracy: 54.124%	Accuracy: **59.278%** Loss: 0.691 Precision: 56.198% Recall/Sensitivity: 72.340% F1-score: 63.256% ROC-AUC: 0.5690	Some overfitting in the later epochs 30, as training performance continues to improve while validation accuracy fluctuates and validation loss begins to increase. The best checkpoint achieved 59.278% validation accuracy, exceeding the majority-class baseline of 51.807%, with a relatively high recall of 72.340% but lower precision of 56.198%. Although data augmentation improved recall and F1-score compared with Model 1, its ROC-AUC of 0.569 indicates weak overall discrimination between benign and malignant cases.

Base CNN + data augmentation + L2 & regularisation [3] (Appendix F.3)	Validation accuracy: 51.546%	Accuracy: **51.546%** Loss: 0.695 Precision: 0.000% Recall: 0.000% F1-score: 0.000% ROC-AUC: 0.508	The results indicate that the model failed to learn meaningful features as both training and accuracy plateaued at approximately 51.5% while the loss remained close to 0.693.
(Base) CNN + data augmentation + hyperparameter tuning [4] (Appendix F.4)	Validation accuracy: 60.309%	Accuracy: **62.887%** Loss: 0.653 Precision: 57.857% Recall: 86.170% F1-score: 69.231% ROC-AUC: 0.661	The best accuracy achieved among all the custom models. The high recall of 86.170% indicates that it detected most malignant cases, although the lower precision of 57.857% suggests many benign cases were incorrectly classified as malignant. The F1-score of 69.231% and ROC-AUC of 0.661 indicate moderate validation performance.
(Base) CNN + data augmentation + hyperparameter tuning – on **X_test** (Appendix F.5)		Accuracy: **58.144%** Loss: 0.684 Precision: 58.289% Recall/Sensitivity: 46.581% F1-score: 51.781% ROC-AUC: 0.5832	The reported accuracy decreased to 58.144%, with an F1-score of 51.781% and ROC-AUC of 0.5832, indicating weaker performance on unseen data X_test. The recall of 46.581% suggests that more than half of the malignant cases were missed. However, the displayed confusion matrix does not match these reported metrics. The final model achieved 54.839% accuracy, 48.571% precision, 50.000% recall, and an F1-score of 49.275%. It correctly classified 204 benign and 136 malignant cases but misclassified 144 benign cases and missed 136 malignant cases. Overall, it is still demonstrating weak generalisation capability.
(Base) CNN + data augmentation + hyperparameter tuning – on **df_test** (Appendix F.6)		Accuracy: **54.839%** Loss: 0.686 Precision: 48.571% Recall/Sensitivity: 50.000% F1-score: 49.275% ROC-AUC: 0.5534	

Pre-trained models

Pre-trained models			
Models	**Metrics (from the best checkpoint)**	**Metrics (with df_test)**	**Description**
ResNet50 (Appendix G.1)	Accuracy: **62.887%** Loss: 0.669 Precision: 84.375% Recall/Sensitivity: 28.723% F1-score: 42.857%	Accuracy: **66.452%** Loss: 0.611 Precision: 64.035% Recall/Sensitivity: 53.676% F1-score: 58.400%	This model achieved 66.452% accuracy with recall improving from 28.723% to 53.676% with unseen data. However, its precision decreased from 84.375% to 64.035%. Its confusion matrix shows that it

	ROC-AUC: 0.7665	ROC-AUC: 0.7063	classified benign cases more successfully than malignant cases, correctly identifying 266 benign and 146 malignant cases while missing 126 malignant cases. There is some overfitting towards epochs 18.
ResNet50 with best hyperparameters (Appendix G.2)	Accuracy: **74.227%** Loss: 0.554 Precision: 77.500% Recall/Sensitivity: 65.957% F1-score: 71.264% ROC-AUC: 0.7888	Accuracy: **64.516%** Loss: 0.616 Precision: 59.286% Recall/Sensitivity: 61.029% F1-score: 60.145% ROC-AUC: 0.7174	Achieved 74.227% validation accuracy and a ROC-AUC of 0.7888, with no substantial divergence between the training and validation curves. However, its accuracy decreased to 64.516% on unseen data, indicating that hyperparameter tuning did not fully eliminate overfitting to the validation data.
VGG16 (Appendix G.3)	Accuracy: **51.546%** Loss: 0.693 Precision: 0.000% Recall/Sensitivity: 0.000% F1-score: 0.000% ROC-AUC: 0.5000	Accuracy: **56.129%** Loss: 0.689 Precision: 0.000% Recall/Sensitivity: 0.000% F1-score: 0.000% ROC-AUC: 0.5000	The VGG16 model failed to learn meaningful differences between the classes, as its loss remained near 0.693 and its ROC-AUC stayed at 0.500, indicating no ability to discriminate between benign and malignant cases. The model is likely to be severely underfit or that it has collapsed.
VGG16 with best hyperparameters (Appendix G.4)		**Accuracy: 56.129%** Loss: 0.690 Precision: 0.000% Recall/Sensitivity: 0.000% F1-score: 0.000% ROC-AUC: 0.5000	

To summarise, almost all the models surpassed the majority-class baseline on the validation set, with the model 4 achieving the strongest validation performance and reduce most of the overfitting observed in earlier models. The decline in performance on the test sets indicates that the custom models still had limited generalisation, while stronger regularisation caused model 3 to underfit and predict only the majority class.

Among the pre-trained models, the tuned ResNet50 produced the strongest overall results, although its accuracy decreased from 74.227% on validation to 64.516% on the test set showing that overfitting was not fully resolved.

Current Limitation

During this study, we can observe various possibilities that hindered the performance of the models, resulting in undesirable performance.

Firstly, the lack of robust infrastructure for running these deep learning algorithms. Throughout the training and testing phases, the local machine that does the computation experienced lagging and slow processing power despite the “small” dataset. Wang L (2024) has also mentioned that it would be ideal to be supported by hardware with robust graphical processing units and tensor processing units, or capabilities to accelerate the training process to train larger datasets. I observed that while setting my epoch as 50, my machine could not handle this much computation. However, the trade-off is that limiting training to fewer epochs may prevent the model from converging fully, resulting in poorer predictive performance because it has insufficient opportunities to learn the underlying patterns in the training data.

Secondly, the lack of quality data used and poor planning pre-processing methods. The dataset chosen was sufficient for an academia level of study, but insufficient for a complete study that is competitive with published models quoted from the literature papers. It was also a tedious process trying to decipher the entire dataset since the metadata was inconsistent, which require extensive workload to manually annotate them, specifically the part on mapping the local image path to the corresponding image type. For this reason, there is a possibility my pre-processing method is problematic causing inconsistencies.

Thirdly, the wrongful selection and understanding of model’s architecture or complexity. Certain model complexity is compactible for specific environments, specifications and data. Furthermore, there are factors that contributes to the model complexity beyond the framework and optimisation process, other factors namely expressive capacity (the capacity of DL models in complex problems) and effective model complexity (complexity of the functions represented by DL models with specific parameterisation) are essential for constructing an optimised model for learning and generalising (Hu et al., 2021).

6 Conclusion (351 words)

This project explores different DL methods for breast cancer classification using mammography, through the usage of custom CNN models with statistical power and utilising pre-built models ResNet50 and VGG16, to enhance the accuracy and the learning capabilities. The results and evaluation have shown that ResNet50 demonstrated the most promising classification capability, but its moderate test performance indicates that further testing would be required before it could be considered reliable for clinical practice.

Given time constraints within this project, future extension to our current progress is to consider is extending the evaluation to additional variants within the VGG and ResNet families, comparing model complexity against classification performance (e.g. VGG19 and ResNet101). Other pre-trained models like U-Net and their families were heavily researched for mammography projects on breast lesion segmentation and have shown good outcomes, which can experiment on for breast cancer classification case studies.

Outside of binary image classification, we can explore other DL computer vision applications for complex models to learn features. An example being object detection which we can use it for learning about the tumour placement, mass and calcification distribution, contributing to the classifiers' predictive capabilities. Furthermore, we can consider adjusting the current models by implementing best practices with residual connections, batch normalisation and separable convolutions.

Despite the setbacks in our study, DL continues to be widely adopted and acknowledged in the medical field, with strong potential to enhance diagnosis and treatment plan beyond breast cancer. It has revolutionised medicinal imaging tasks given its demonstrated ability to identify and interpret patterns, outperforming legacy detection and diagnosis systems and supporting radiologists with diagnostic decision-making process. However, further trials and research remain necessary, given the limitations of DL such as scarcity of high-quality data available for effective training, data pre-processing techniques and model complexity.

DL is considered as a complementary support tool that improves radiologists' efficiency and workload. The question on whether DL assisted mammography is reliable enough to replace radiologists depends on its ability to demonstrate consistent accuracy, ability to generalise, transparency, and safety across diverse populations and real-world clinical settings, making human expertise and intervention still relevant at present.

Reference

Chollet, F. (2021) Deep Learning with Python. 2nd edn. Shelter Island, NY: Manning Publications.

A brief history of cancer. (n.d.). American Cancer Society. https://www.cancer.org/cancer/understanding-cancer/history-of-cancer.html

Nass, S. J., Henderson, I. C., & Lashof, J. C. (2001). 1 Introduction. Mammography and Beyond: Developing Technologies for the Early Detection of Breast Cancer - NCBI Bookshelf. https://www.ncbi.nlm.nih.gov/books/NBK222338/

Center for Devices and Radiological Health. (n.d.). MQSA Insights > Mammography Equipment Evaluations and the Annual Survey - Is your equipment up to the task? FDA. https://wayback.archive-it.org/7993/20170112093812/http://www.fda.gov/Radiation-EmittingProducts/MammographyQualityStandardsActandProgram/FacilityScorecard/ucm113668.htm

Ślusarczyk-Kacprzyk, W., Skrzyński, W., & Fabiszewska, E. (2016). Evaluation of Doses and Image Quality in Mammography with Screen-Film, CR, and DR Detectors – Application of the ACR Phantom. Polish Journal of Radiology, 81, 386–391. https://doi.org/10.12659/pjr.897304

Nicosia, L., Gnocchi, G., Gorini, I., Venturini, M., Fontana, F., Pesapane, F., Abiuso, I., Bozzini, A. C., Pizzamiglio, M., Latronico, A., Abbate, F., Meneghetti, L., Battaglia, O., Pellegrino, G., & Cassano, E. (2023). History of Mammography: Analysis of Breast Imaging Diagnostic Achievements over the Last Century. Healthcare, 11(11), 1596. https://doi.org/10.3390/healthcare11111596

Wang L. Mammography with deep learning for breast cancer detection. Front Oncol. 2024 Feb 12;14:1281922. doi: 10.3389 https://doi.org/10.3389/fonc.2024.1281922 PMID: 38410114; PMCID: PMC10894909

Example of a digital breast tomosynthesis (DBT) image of a breast. . . (n.d.). ResearchGate. https://www.researchgate.net/figure/Example-of-a-digital-breast-tomosynthesis-DBT-image-of-a-breast-phantom-with-simulated_fig3_370560178

Advances in Breast Imaging: Evolution & History of Mammography | Radiology. (n.d.). https://radiology.ucsf.edu/news/advances-breast-imaging-evolution-history-mammography

New breast imaging modalities in the assessment of abnormal breast findings. (2023, June 1). SingHealth. https://www.singhealth.com.sg/news/defining-med/new-breast-imaging-modalities

Atlas of breast cancer early detection. (n.d.). https://screening.iarc.fr/atlasbreastdetail.php?Index=018&e=

Van Der Steeg, A. F. W., Keyzer-Dekker, C. M. G., De Vries, J., & Roukema, J. A. (2010). Effect of abnormal screening mammogram on quality of life. British Journal of Surgery, 98(4), 537–542. https://doi.org/10.1002/bjs.7371

Uscher, J. (2026, June 22). Mammograms: What you need to know. https://www.breastcancer.org/screening-testing/mammograms

Breastcancer.org. (2026, May 18). Understanding breast calcifications. https://www.breastcancer.org/screening-testing/mammograms/calcifications

Fenton, J. J., Taplin, S. H., Carney, P. A., Abraham, L., Sickles, E. A., D'Orsi, C., Berns, E. A., Cutter, G., Hendrick, R. E., Barlow, W. E., & Elmore, J. G. (2007). Influence of Computer-Aided Detection on performance of screening mammography. New England Journal of Medicine, 356(14), 1399–1409. https://doi.org/10.1056/nejmoa066099

CBIS-DDSM: Breast Cancer Image Dataset. (2021, January 24). https://www.kaggle.com/datasets/awsaf49/cbis-ddsm-breast-cancer-image-dataset

Sawyer-Lee, R., Gimenez, F., Hoogi, A., & Rubin, D. (2016). Curated Breast Imaging Subset of Digital Database for Screening Mammography (CBIS-DDSM) [Data set]. The Cancer Imaging Archive. https://doi.org/10.7937/K9/TCIA.2016.7O02S9CY

Lee, R. S., Gimenez, F., Hoogi, A., Miyake, K. K., Gorovoy, M., & Rubin, D. L. (2017). A curated mammography data set for use in computer-aided detection and diagnosis research. Scientific Data, 4(1), 170177. https://doi.org/10.1038/sdata.2017.177

Guirguis, M. S., Adrada, B., Santiago, L., Candelaria, R., & Arribas, E. (2021). Mimickers of breast malignancy: imaging findings, pathologic concordance and clinical management. Insights Into Imaging, 12(1), 53. https://doi.org/10.1186/s13244-021-00991-x

Drukteinis, J. S., Mooney, B. P., Flowers, C. I., & Gatenby, R. A. (2013). Beyond Mammography: New frontiers in breast cancer screening. The American Journal of Medicine, 126(6), 472–479. https://doi.org/10.1016/j.amjmed.2012.11.025

Abeelh, E. A., & AbuAbeileh, Z. (2024). Comparative effectiveness of mammography, ultrasound, and MRI in the detection of breast carcinoma in dense breast tissue: a systematic review. Cureus, 16(4), e59054. https://doi.org/10.7759/cureus.59054

Chen, H., Zhou, J., Chen, Q., & Deng, Y. (2021). Comparison of the sensitivity of mammography, ultrasound, magnetic resonance imaging and combinations of these imaging modalities for the detection of small (≤2 cm) breast cancer. Medicine, 100(26), e26531. https://doi.org/10.1097/md.0000000000026531

Jaglan, P., Dass, R., & Duhan, M. (2019). Breast cancer detection techniques: issues and challenges. Journal of the Institution of Engineers (India) Series B, 100(4), 379–386. https://doi.org/10.1007/s40031-019-00391-2

Lee, K. (2010). Optical mammography: Diffuse optical imaging of breast cancer. World Journal of Clinical Oncology, 2(1), 64. https://doi.org/10.5306/wjco.v2.i1.64

Verma, P., Chaudhary, S., Sharma, M., Agarwal, S., & Rajawat, V. (2025, December). Comprehensive analysis of breast cancer detection technologies. In Proceedings of the 7th International Conference on Information Management & Machine Intelligence, 1–8. ACM. https://doi.org/10.1145/3793449.3793463

Altoe, M. L., Marone, A., Kim, H. K., Kalinsky, K., Hershman, D. L., Hielscher, A. H., & Ha, R. S. (2019). Diffuse optical tomography of the breast: a potential modifiable biomarker of breast cancer risk with neoadjuvant chemotherapy. Biomedical Optics Express, 10(8), 4305. https://doi.org/10.1364/boe.10.004305

Giger, M. L., Chan, H., & Boone, J. (2008). Anniversary Paper: History and status of CAD and quantitative image analysis: The role ofMedical Physicsand AAPM. Medical Physics, 35(12), 5799–5820. https://doi.org/10.1118/1.3013555

Nasir, F., Rahman, S., & Nasir, N. (2025). Breast Cancer Detection Using Convolutional Neural Networks: A Deep Learning-Based approach. Cureus, 17(5), e83421. https://doi.org/10.7759/cureus.83421

HyFormer-Net: A Synergistic CNN-Transformer with Interpretable Multi-Scale Fusion for Breast Lesion Segmentation and Classification in Ultrasound Images. (Mohammad Amanour Rahman, 2025). https://arxiv.org/html/2511.01013v1

Optimized Residual Attention U-Net-Based lung cancer detection. (n.d.). IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/document/10847385

Abdelhafiz, D., Nabavi, S., Ammar, R., Yang, C., & Bi, J. (2019). Residual deep learning system for mass segmentation and classification in mammography. In Proceedings of the 10th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics (pp. 475–484). Association for Computing Machinery. https://doi.org/10.1145/3307339.3342157

Huang, Y., & Huang, C. (2025). Liver Tumor Classification and Segmentation from CT Images with a Trans RU-Net and EfficientNet Hybrid System. Lecture Notes in Electrical Engineering, 149–154. https://doi.org/10.1007/978-981-96-2395-2_26

Ha, R., Chang, P., Karcich, J., Mutasa, S., Van Sant, E. P., Liu, M. Z., & Jambawalikar, S. (2018). Convolutional neural network based breast cancer risk stratification using a mammographic dataset. Academic Radiology, 26(4), 544–549. https://doi.org/10.1016/j.acra.2018.06.020

Abdel Rahman, A. S., Belhaouari, S. B., Bouzerdoum, A., Baali, H., Alam, T., & Eldaraa, A. M. (2020). Breast mass tumor classification using deep learning. In 2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT) (pp. 271–276). IEEE. https://doi.org/10.1109/ICIoT48696.2020.9089535

Wang, Y., Choi, E. J., Choi, Y., Zhang, H., Jin, G. Y., & Ko, S. (2020). Breast Cancer Classification in Automated Breast Ultrasound Using Multiview Convolutional Neural Network with Transfer Learning. Ultrasound in Medicine & Biology, 46(5), 1119–1132. https://doi.org/10.1016/j.ultrasmedbio.2020.01.001

Salama, W. M., & Aly, M. H. (2021). Deep learning in mammography images segmentation and classification: Automated CNN approach. Alexandria Engineering Journal, 60(5), 4701–4709. https://doi.org/10.1016/j.aej.2021.03.048

Hu, X., Chu, L., Pei, J., Liu, W., & Bian, J. (2021). Model complexity of deep learning: a survey. Knowledge and Information Systems, 63(10), 2585–2619. https://doi.org/10.1007/s10115-021-01605-0

Nanni, L., Brahnam, S., Paci, M., & Ghidoni, S. (2022). Comparison of different convolutional neural network activation functions and methods for building ensembles for small to midsize medical data sets. Sensors, 22(16), 6129. https://doi.org/10.3390/s22166129

TensorFlow. (2024, August 16). Convolutional neural network (CNN). https://www.tensorflow.org/tutorials/images/cnn#:~:text=Dense%20layers%20take,layers%20on%20top.

He, K., Zhang, X., Ren, S., & Sun, J. (2015, December 10). Deep residual learning for image recognition. arXiv.org. https://arxiv.org/abs/1512.03385?ref=blog.roboflow.com

Appendices

Appendix A – Gantt chart

FYP%20Gantt%20Chart%20-%20June.xlsx

Appendix B – Project pipeline flowchart



FYP Flowchart.jpg

Key elements	Description
Input dataset	Dataset for this project will be using Digital Database for Screening Mammography (DDSM), containing samples of mammogram images
Data collection	Collect the dataset from the source.
Data preprocessing	Dataset to undergo transformation (e.g. standardisation, normalisation, remove unwanted features, encoding, handle missing values)
Data splitting	80% training set, 10% validation set, 10% test set
Choose model	At this current stage, the default model would be the CNN
Model training/validation	Model learns to make correct predictions using training and validation dataset, while also assessing the effectiveness of the model
Results of performance metrics/hyperparameter tuning/deployment	To use appropriate performance metrics for measuring model's predictive accuracy by balancing precision and recall Based on the results, we can determine whether the model is suitable for deployment. If otherwise, the hyperparameter will be adjusted to make improvements to the model

Appendix C – FYP diagrams/recording slide



Appendix D – Measure of Success metrics description

Performance metrics	Description	Formula (%)
Accuracy	Measures the correct predictions made by the classification model out of all predictions. **Baseline's accuracy score** – with reference to the textbook (Chollet, 2021, p.136-137), this evaluation method is called "common-sense baseline", which represents the performance of a simplest random	Accuracy = **Correct Predictions** (True Positive + True Negative) / **Total Predictions** (True Positive + True Negative + False Positive + False Negative)

	classifier, and when our model's accuracy exceeds the baseline's accuracy, it is an indicator that the model is being productive, in the sense that it is making predictions that generalise based on the input data.	
Loss	Measures how well the predictions made by the model match with the actual true label.	Binary Cross-Entropy
AUC	Measures the model's diagnostic or distinguish power between the two classes (Benign, Malignant) across all possible probability thresholds. '1.0' AUC score is the benchmark for optimal performing model.	$\text{AUC} = \int_0^1 \text{T PR}\left(\text{FPR}^{-1}(\text{t})\right) \text{dt}$ Where True Positive Rate (TRP) = True Positive / (True Positive + False Negative) Where False Positive Rate (FPR) = False Positive / (False Positive + True Negative)
F1 score	For calculating and handling class imbalance, by penalising extreme gap between precision and recall.	$\text{F}_1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ Where Precision = True Positive / (True Positive + False Positive) Where Recall = True Positive / (True Positive + False Negative)
Precision	Measures the precision of the classifier.	Precision = True Positive / (True Positive + False Positive)
Sensitivity/Recall	Measures the model's ability to correctly classify actual true positive cases with minimised false negatives.	Sensitivity = True Positive / (True Positive + False Negative)

Appendix E – CNN models design process

Custom CNN models	
Layers/Hyperparameter	**Description**
inputs = keras.Input(shape=(224, 224, 1))	Since we resized the image during the early data pre-processing step, we assign the input tensor for the CNN model to be in the same dimension with a single input channel since the raw data only has one colour channel (grayscale).
Conv2d layer filters=32, kernel_size=3, activation="relu"	Also known as a combination of convolution and activation layer, the conv2d layer is made up of convolution stacks that can process and transform 2D image data to learn spatially local patterns. Each layer uses different range of filters (32 to 256) for the model to detect image features (e.g. edges, textures, curves, boundaries, variations, etc). A 3 by 3 kernel was selected as it is sufficient to capture local patterns efficiently. The activation function chosen is ReLU as it is commonly used in classification problems using convolution layers or hidden dense layers for introducing non-linearity to classify complex data with non-linear relationships (Nanni et al., 2022). It is also computationally efficient and helps reduce the vanishing-gradient problem during training.
MaxPooling2d layer pool_size=2	Also known as pooling layer, it applies a 2 by 2 filter that can downsize the spatial dimensions

	across the feature map, helping to reduce computation while preventing overfitting.
Model.add(layers.Flatten())	Convnets require a layer for reshaping spatial feature maps before introducing feature learning to achieve classification or regression. Flatten() was chosen to transform multi-dimensional feature maps into one-dimension vector, to pass extracted features to the subsequent dense layers for binary classification. Dense layers take vectors as input (which are 1D), while the current output is a 3D tensor (TensorFlow, 2024).
Fully connected layers model.add(layers.Dense(64, activation='relu'))	These layers form the classifier The 64-neuron dense layer combines the extracted image features and uses ReLU activation to learn non-linear relationships.
outputs = layers.Dense(1, activation="sigmoid")(x) #single output binary	The final dense layer contains one neuron with sigmoid activation, producing a probability between 0 and 1. Using a classification threshold of 0.5, probabilities below 0.5 are classified as benign (class 0), while probabilities of 0.5 or above are classified as malignant (class 1).
.compile(loss="binary_crossentropy", optimizer="adam",metrics=["accuracy"])	Since we are using a binary classifier to classify between two classes, the model was compiled using binary cross-entropy as the loss function. Binary cross-entropy measures the difference between the true binary labels and the probabilities produced by the sigmoid output layer, providing the error signal used during training.

	The Adam optimiser was selected to update the model's weights because it adaptively adjusts the learning rate for each parameter, supporting efficient training. We already declared accuracy as the primary metric in the previous sub-section. Hence, it was included as an evaluation metric for compilation.
ResNet50/VGG16 model	

def baseResNet50():
 	
  baseRN50 = keras.applications.ResNet50(
  weights="imagenet",
  include_top=False,
  input_shape=(224, 224, 3),
  name="base_resnet50",
)
  return baseRN50
 	
def baseVGG16():
  baseVGG = keras.applications.vgg16.VGG16(
  weights="imagenet",
  include_top=False,
  input_shape=(224, 224, 3),
  name="base_vgg16",
)
 	
  return baseVGG
	Define the pre-trained ResNet50/VGG16 feature extractor.

x = layers.Concatenate(axis=-1)([x, x, x])	Because ResNet50 expects an input with three colour channels, this layer will copy the grayscale channel three times, converting the shape from 224 x 224 x 1 to 224 x 224 x 3. It only makes the images compatible with the pre-trained network without adding any colour properties.

x = preprocess_input(x) x = vgg16_preprocess_input(x)	This applies the preprocessing used when ResNet50/VGG16 was originally trained on ImageNet, ensuring the input mammograms are presented to the pre-trained network in a compatible numerical format.
x = resnetBase(x, training=False) x = vgg16Base(x, training=False)	For passing images through the ResNet50/VGG16 convolutional base for feature extraction.
x = layers.GlobalAveragePooling2D()(x) [FOR RESNET50]	Helps to convert feature maps into vectors, reduce overfitting, and pass extracted features to the subsequent dense layers for binary classification.
.compile(optimizer="rmsprop"...	RMSprop was selected as the optimiser because it adapts the learning rate for each trainable parameter. This helps stabilise weight updates when the size of the model's weight updates changes during training.

Appendix F.1 – Base CNN Model results

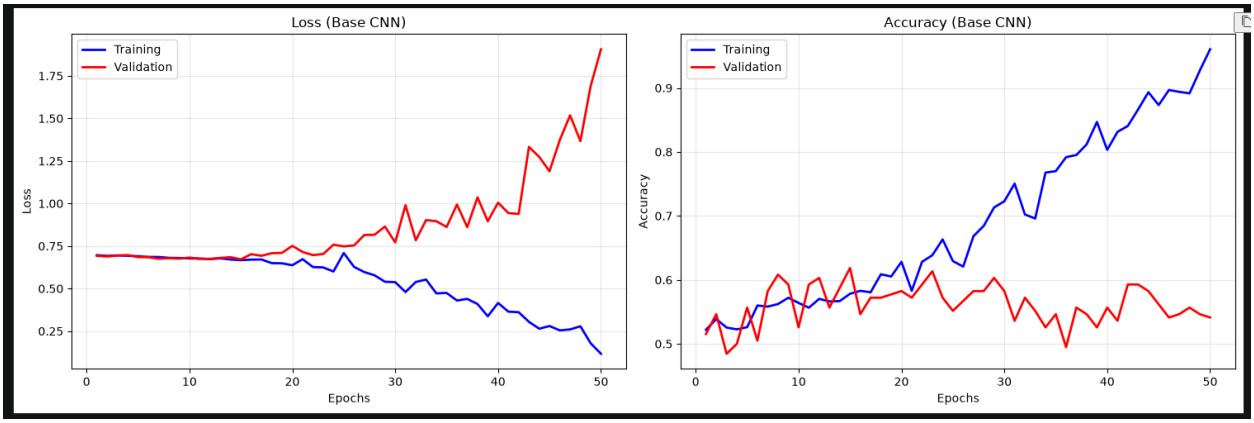
Model: "base_cnn"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_4 (Conv2D)	(None, 10, 10, 256)	590,080
flatten (Flatten)	(None, 25600)	0
dense (Dense)	(None, 64)	1,638,464
dense_1 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

Trainable params: 2,616,449 (9.98 MB)

Non-trainable params: 0 (0.00 B)



Appendix F.2 – Base + Data Augmentation Model results

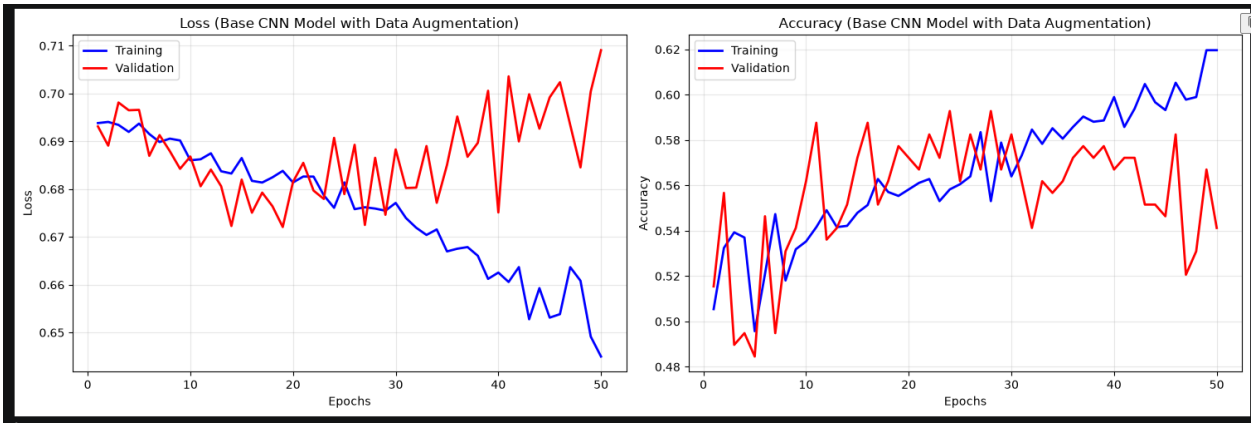
Model: "aug_cnn"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 224, 224, 1)	0
mammogram_augmentation (Sequential)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, [illegible], [illegible], 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_4 (Conv2D)	(None, [illegible], [illegible], 256)	590,080
flatten_2 (Flatten)	(None, 25600)	0
dense_4 (Dense)	(None, 64)	1,638,464
dense_5 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

Trainable params: 2,616,449 (9.98 MB)

Non-trainable params: 0 (0.00 B)



Appendix F.3 – Base + Data Augmentation + L2 Regularisation Model results

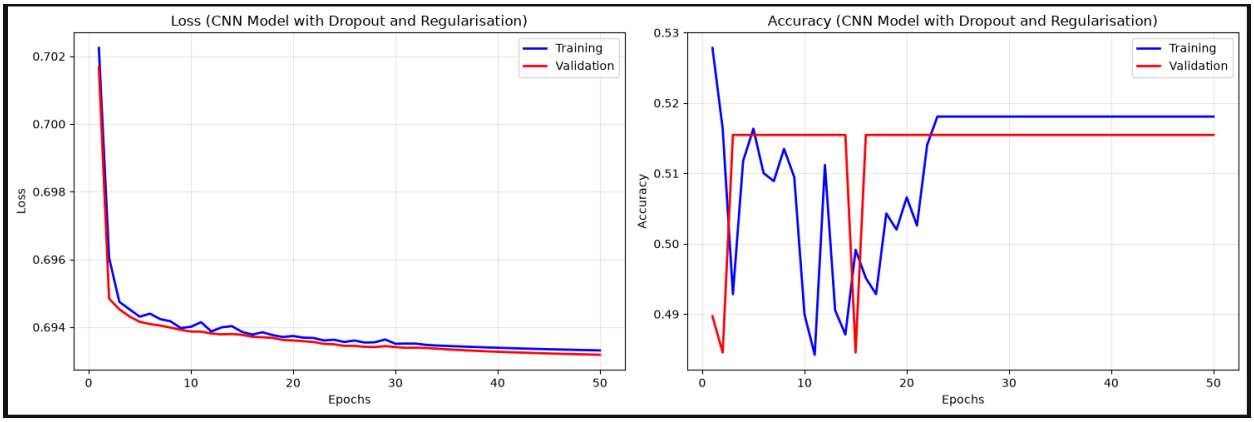
Model: "l2reg_cnn"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 224, 224, 1)	0
mammogram_augmentation (Sequential)	(None, 224, 224, 1)	0
conv2d_10 (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d_8 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_11 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_9 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_12 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_10 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_13 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_11 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_14 (Conv2D)	(None, 10, 10, 256)	590,080
flatten_2 (Flatten)	(None, 25600)	0
dense_4 (Dense)	(None, 64)	1,638,464
dropout (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

Trainable params: 2,616,449 (9.98 MB)

Non-trainable params: 0 (0.00 B)



Appendix F.4 – Base + Data Augmentation + Hyperparameter Tuning Model results

Model: "tuned_cnn"

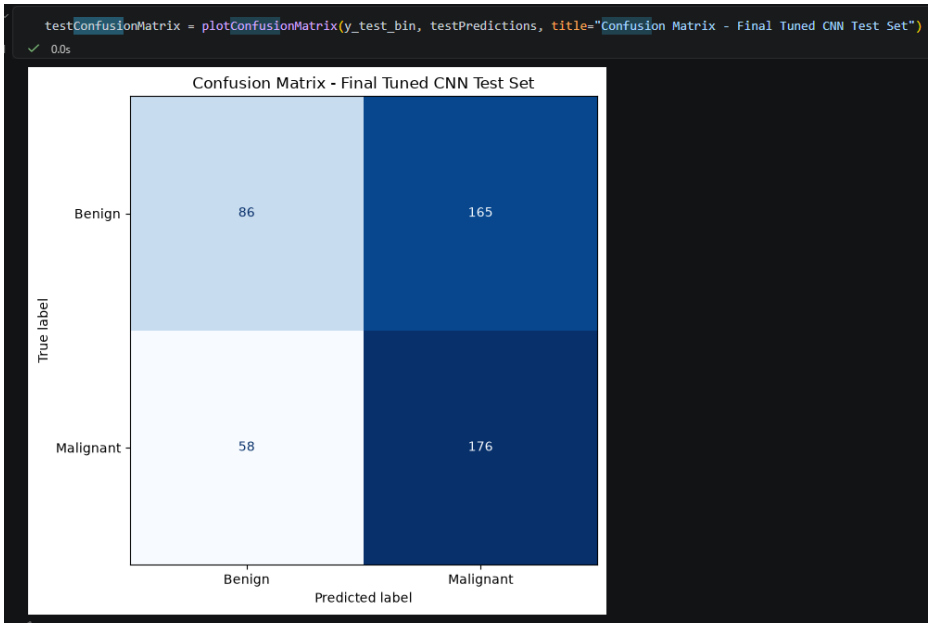
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 1)	0
mammogram_augmentation (Sequential)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_4 (Conv2D)	(None, 10, 10, 256)	590,080
flatten (Flatten)	(None, 25600)	0
dense (Dense)	(None, 64)	1,638,464
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

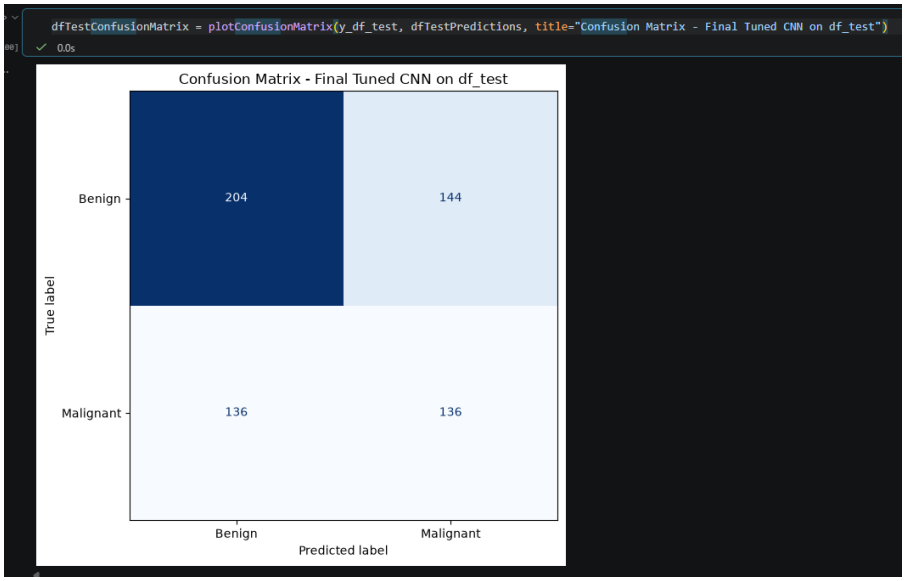
Trainable params: 2,616,449 (9.98 MB)

Non-trainable params: 0 (0.00 B)

Appendix F.5 – Base + Data Augmentation + Hyperparameter Tuning Model results – on **X_test**



Appendix F.6 – Base + Data Augmentation + Hyperparameter Tuning Model results – on **df_test**



Appendix G.1 – ResNet50 base

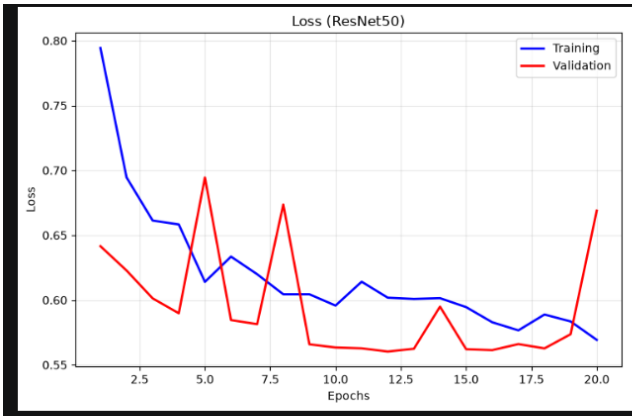
Model: "resnet50_feature_extract"

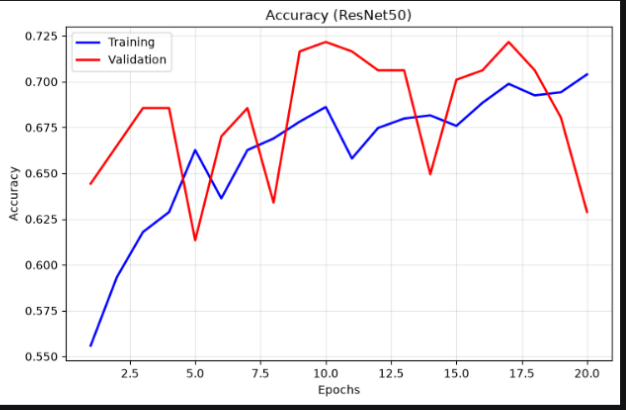
Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 1)	0	-
mammogram_augmenta… (Sequential)	(None, 224, 224, 1)	0	input_layer_1[0]…
concatenate (Concatenate)	(None, 224, 224, 3)	0	mammogram_augmen… mammogram_augmen… mammogram_augmen…
rescaling (Rescaling)	(None, 224, 224, 3)	0	concatenate[0][0]
get_item (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_1 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_2 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
stack (Stack)	(None, 224, 224, 3)	0	get_item[0][0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 224, 224, 3)	0	stack[0][0]
base_resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712	add[0][0]
global_average_poo… (GlobalAveragePool…	(None, 2048)	0	base_resnet50[0]…
dropout (Dropout)	(None, 2048)	0	global_average_p…
dense (Dense)	(None, 1)	2,049	dropout[0][0]

Total params: 23,589,761 (89.99 MB)

Trainable params: 2,049 (8.00 KB)

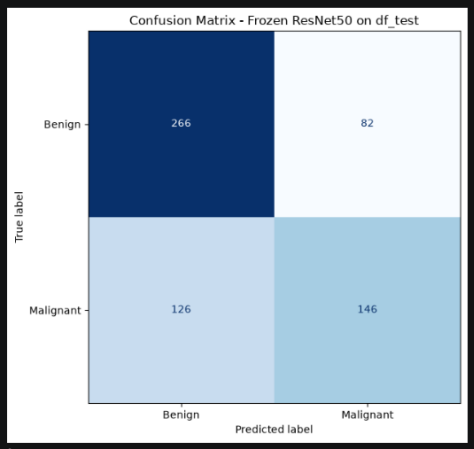
Non-trainable params: 23,587,712 (89.98 MB)





========== ResNet50 ==========
Accuracy: 62.887%
Loss: 0.669
Precision: 84.375%
Recall/Sensitivity: 28.723%
F1-score: 42.857%
ROC-AUC: 0.7665

```
============== ResNet50 Frozen Feature Extraction - df_test ==============
Accuracy: 66.452%
Loss: 0.611
Precision: 64.035%
Recall/Sensitivity: 53.676%
F1-score: 58.400%
ROC-AUC: 0.7063
```




```
array([[266,  82],
       [126, 146]])
```

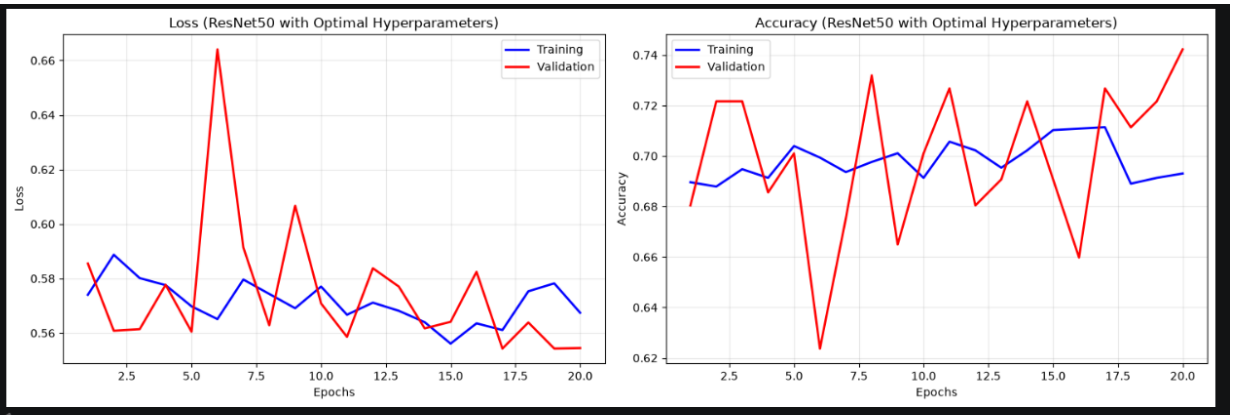
Appendix G.2 – ResNet50 with best hyperparameters

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 224, 224, 1)	0	-
mammogram_augmenta… (Sequential)	(None, 224, 224, 1)	0	input_layer_2[0]…
concatenate_1 (Concatenate)	(None, 224, 224, 3)	0	mammogram_augmen… mammogram_augmen… mammogram_augmen…
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0	concatenate_1[0]…
get_item_3 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
get_item_4 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
get_item_5 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
stack_1 (Stack)	(None, 224, 224, 3)	0	get_item_3[0][0], get_item_4[0][0], get_item_5[0][0]
add_1 (Add)	(None, 224, 224, 3)	0	stack_1[0][0]
base_resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712	add_1[0][0]
global_average_poo… (GlobalAveragePool…	(None, 2048)	0	base_resnet50[1]…
dense_1 (Dense)	(None, 64)	131,136	global_average_p…
dropout_1 (Dropout)	(None, 64)	0	dense_1[0][0]
dense_2 (Dense)	(None, 1)	65	dropout_1[0][0]

```
Total params: 23,718,913 (90.48 MB)

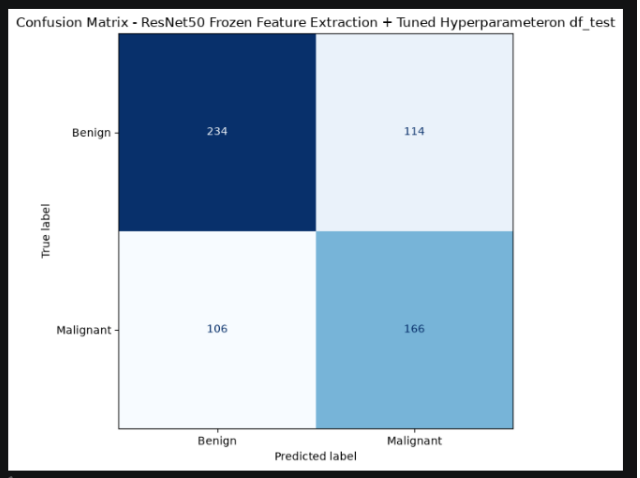
Trainable params: 131,201 (512.50 KB)

Non-trainable params: 23,587,712 (89.98 MB)
```




```
============== ResNet50 ==============
Accuracy: 74.227%
Loss: 0.554
Precision: 77.500%
Recall/Sensitivity: 65.957%
F1-score: 71.264%
ROC-AUC: 0.7888
ResNet50 frozen: True
Trainable ResNet50 weights: 0
```

```
============== ResNet50 Frozen Feature Extraction + Tuned Hyperparameter - df_test ==============
Accuracy: 64.516%
Loss: 0.616
Precision: 59.286%
Recall/Sensitivity: 61.029%
F1-score: 60.145%
ROC-AUC: 0.7174
```




```
array([[234, 114],
       [106, 166]])
```

Appendix G.3 – VGG16

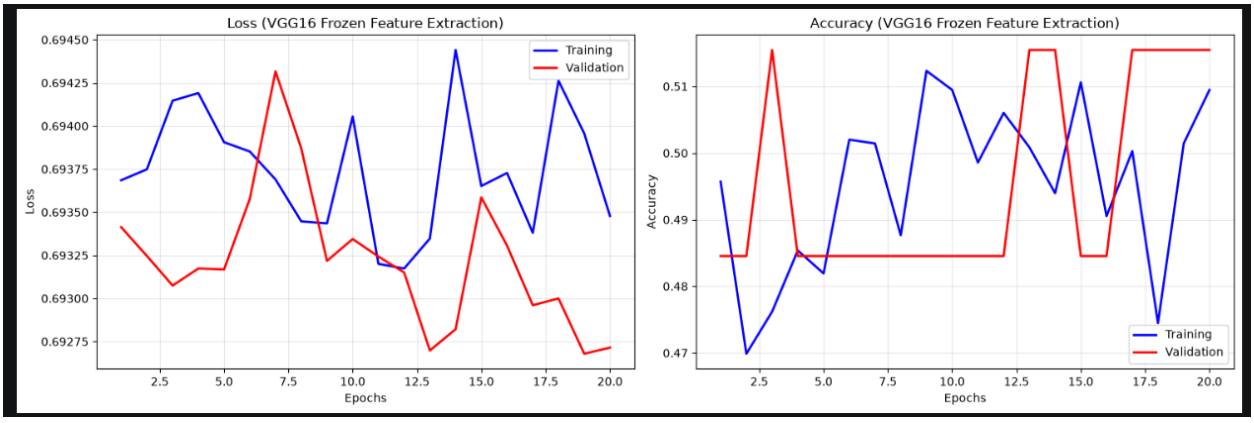
Model: "vgg16_feature_extract"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 1)	0	-
mammogram_augmenta… (Sequential)	(None, 224, 224, 1)	0	input_layer_1[0]…
concatenate (Concatenate)	(None, 224, 224, 3)	0	mammogram_augmen… mammogram_augmen… mammogram_augmen…
rescaling (Rescaling)	(None, 224, 224, 3)	0	concatenate[0][0]
get_item (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_1 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
get_item_2 (GetItem)	(None, 224, 224)	0	rescaling[0][0]
stack (Stack)	(None, 224, 224, 3)	0	get_item[0][0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 224, 224, 3)	0	stack[0][0]
base_vgg16 (Functional)	(None, 7, 7, 512)	14,714,688	add[0][0]
flatten (Flatten)	(None, 25088)	0	base_vgg16[0][0]
dense (Dense)	(None, 256)	6,422,784	flatten[0][0]
dropout (Dropout)	(None, 256)	0	dense[0][0]
dense_1 (Dense)	(None, 1)	257	dropout[0][0]

Total params: 21,137,729 (80.63 MB)

Trainable params: 21,137,729 (80.63 MB)

Non-trainable params: 0 (0.00 B)

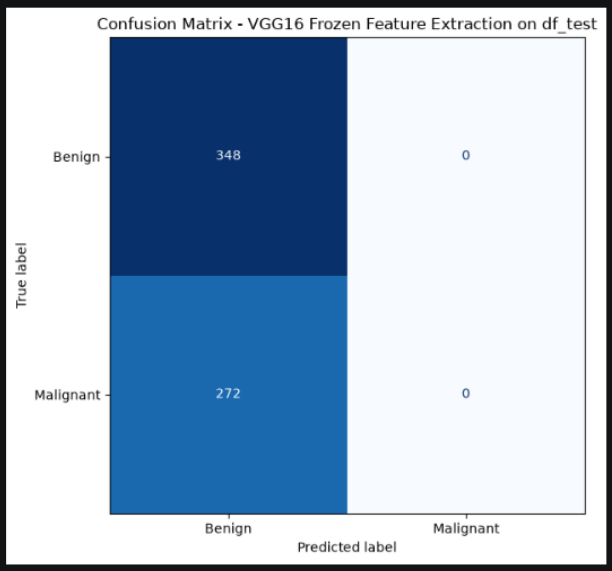


```
featureExtractionResults = printPerfMetrics(vgg16Model, "VGG16", dataset=validation_dataset, y_true=y_val_bin)
```

13.6s

```
=============== VGG16 ===============
Accuracy: 51.546%
Loss: 0.693
Precision: 0.000%
Recall/Sensitivity: 0.000%
F1-score: 0.000%
ROC-AUC: 0.5000
```

```
============== VGG16 Frozen Feature Extraction - df_test ==============
Accuracy: 56.129%
Loss: 0.689
Precision: 0.000%
Recall/Sensitivity: 0.000%
F1-score: 0.000%
ROC-AUC: 0.5000
```




```
array([[348,   0],
       [272,   0]])
```

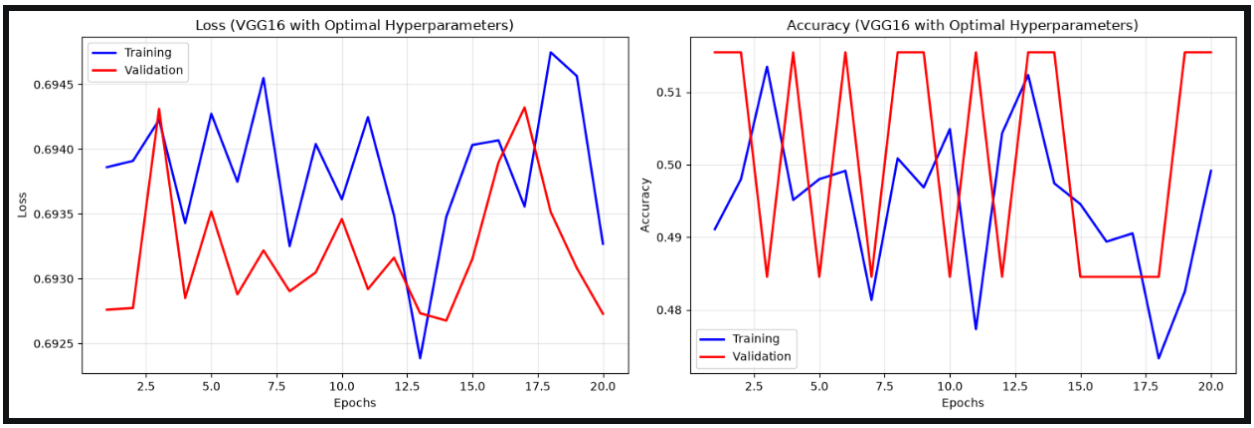
Appendix G.4 – VGG16 with best hyperparameters

Model: "vgg16_op_hyperparameters"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 224, 224, 1)	0	-
mammogram_augmenta… (Sequential)	(None, 224, 224, 1)	0	input_layer_2[0]…
concatenate_1 (Concatenate)	(None, 224, 224, 3)	0	mammogram_augmen… mammogram_augmen… mammogram_augmen…
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0	concatenate_1[0]…
get_item_3 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
get_item_4 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
get_item_5 (GetItem)	(None, 224, 224)	0	rescaling_1[0][0]
stack_1 (Stack)	(None, 224, 224, 3)	0	get_item_3[0][0], get_item_4[0][0], get_item_5[0][0]
add_1 (Add)	(None, 224, 224, 3)	0	stack_1[0][0]
base_vgg16 (Functional)	(None, 7, 7, 512)	14,714,688	add_1[0][0]
flatten_1 (Flatten)	(None, 25088)	0	base_vgg16[1][0]
dense_2 (Dense)	(None, 64)	1,605,696	flatten_1[0][0]
dropout_1 (Dropout)	(None, 64)	0	dense_2[0][0]
dense_3 (Dense)	(None, 1)	65	dropout_1[0][0]

Total params: 16,320,449 (62.26 MB)

Trainable params: 16,320,449 (62.26 MB)

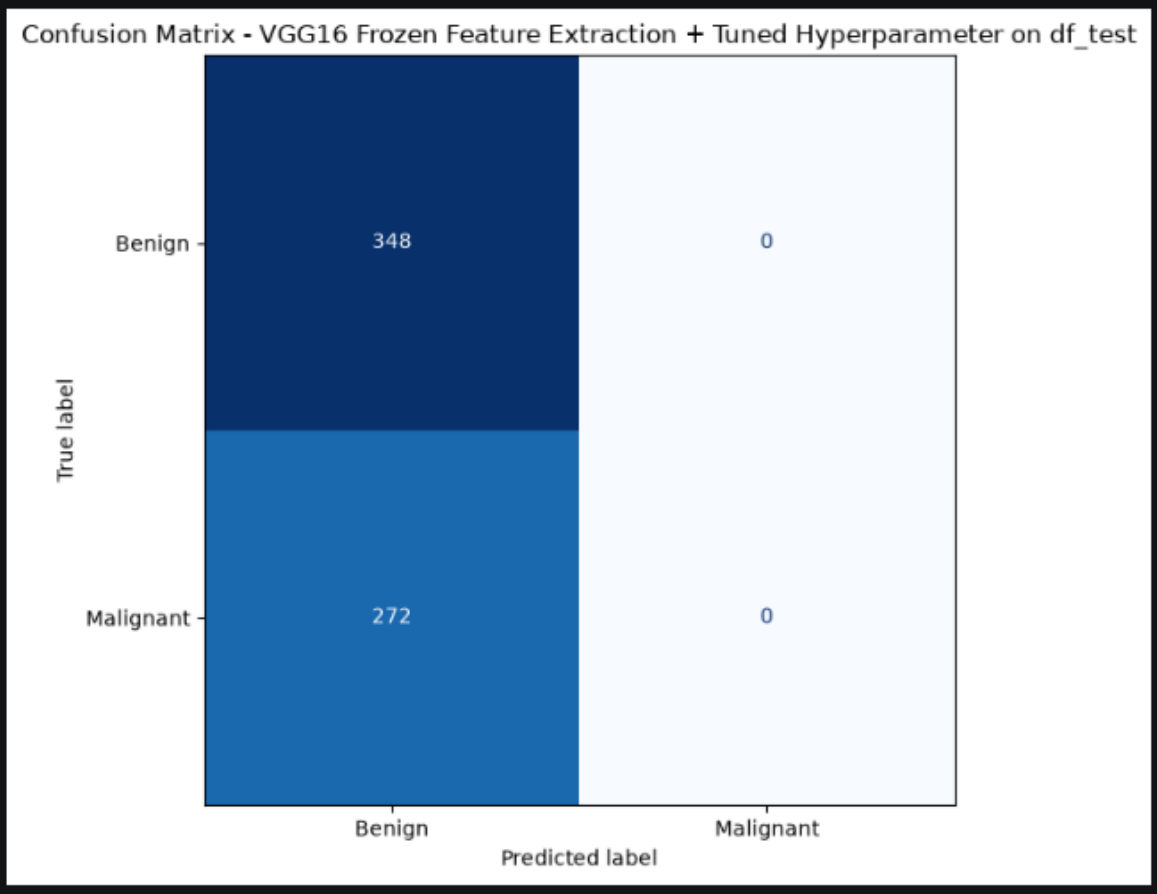



```
vggValidationResults = printPerfMetrics(best_hp_vgg16, "VGG16 Frozen Feature Extraction + Tuned Hyperparameter - validation set", dataset=vggValidationDataset, y_true=y_val_bin)
```

✓ 33.9s

```
========== VGG16 Frozen Feature Extraction + Tuned Hyperparameter - validation set ==========
Accuracy: 51.546%
Loss: 0.693
Precision: 0.000%
Recall/Sensitivity: 0.000%
F1-score: 0.000%
ROC-AUC: 0.5000
```

```
=============== VGG16 Frozen Feature Extraction + Tuned Hyperparameter - df_test ===============
Accuracy: 56.129%
Loss: 0.690
Precision: 0.000%
Recall/Sensitivity: 0.000%
F1-score: 0.000%
ROC-AUC: 0.5000
```




```
array([[348,   0],
       [272,   0]])
```

Appendix H – Codebase in HTML & Jupyter Notebook


main.html


main.ipynb